



FAKULTA APLIKOVANÝCH VĚD
ZÁPADOČESKÉ UNIVERZITY
V PLZNI

KATEDRA INFORMATIKY
A VÝPOČETNÍ TECHNIKY

Diplomová práce

Automatické generování end-to-end testů pomocí velkých jazykových modelů (LLM)

Bc. Milan Janoch



FAKULTA APLIKOVANÝCH VĚD
ZÁPADOČESKÉ UNIVERZITY
V PLZNI

KATEDRA INFORMATIKY
A VÝPOČETNÍ TECHNIKY

Diplomová práce

Automatické generování end-to-end testů pomocí velkých jazykových modelů (LLM)

Bc. Milan Janoch

Vedoucí práce

Ing. Martin Dostal, Ph.D.

© Bc. Milan Janoch, 2026.

Všechna práva vyhrazena. Žádná část tohoto dokumentu nesmí být reprodukována ani rozšiřována jakoukoli formou, elektronicky či mechanicky, fotokopírováním, nahráváním nebo jiným způsobem, nebo uložena v systému pro ukládání a vyhledávání informací bez písemného souhlasu držitelů autorských práv.

Citace v seznamu literatury:

JANOCH, Bc. Milan. *Automatické generování end-to-end testů pomocí velkých jazykových modelů (LLM)*. Plzeň, 2026. Diplomová práce. Západočeská univerzita v Plzni, Fakulta aplikovaných věd, Katedra informatiky a výpočetní techniky. Vedoucí práce Ing. Martin Dostal, Ph.D.

Podklad pro zadání DIPLOMOVÉ práce studenta

Jméno a příjmení: **Bc. Milan JANOCH**

Osobní číslo: **A24N0104P**

Adresa: **V Brance 20, Přeštice, 33401 Přeštice, Česká republika**

Téma práce: **Automatické generování end-to-end testů pomocí velkých jazykových modelů (LLM)**

Téma práce anglicky: **Automatic generation of end-to-end tests using large language models (LLM)**

Jazyk práce: **Čeština**

Související osoby: **Ing. Martin Dostal, Ph.D. (Vedoucí)**

Katedra informatiky a výpočetní techniky

Zásady pro vypracování:

1. Prostudujte současné možnosti generování kódu pomocí velkých jazykových modelů (LLM), se zvláštním zaměřením na oblast automatizovaného testování softwaru.
2. Analyzujte strukturu uživatelských příběhů a specifikací běžně používaných v nástrojích pro řízení vývoje softwaru a identifikujte klíčové prvky využitelné pro generování testovacích scénářů.
3. Navrhněte architekturu systému pro automatické generování end-to-end (E2E) testů z uživatelských příběhů s využitím LLM.
4. Implementujte funkční prototyp navrženého řešení využívající vybraný LLM a vhodný framework pro E2E testování.
5. Otestujte funkčnost systému na vzorových scénářích a vyhodnoťte kvalitu generovaných testů.
6. Kriticky zhodnoťte přínosy a limity řešení a navrhněte možnosti dalšího rozšíření.

Seznam doporučené literatury:

Dodá vedoucí diplomové práce

Podpis studenta:

Datum:

Podpis vedoucího práce:

Datum:

Prohlášení

Prohlašuji, že jsem tuto diplomovou práci vypracoval samostatně a výhradně s použitím citovaných pramenů, literatury a dalších odborných zdrojů. Tato práce nebyla využita k získání jiného nebo stejného akademického titulu.

Beru na vědomí, že se na moji práci vztahují práva a povinnosti vyplývající ze zákona č. 121/2000 Sb., autorského zákona v platném znění, zejména skutečnost, že Západočeská univerzita v Plzni má právo na uzavření licenční smlouvy o užití této práce jako školního díla podle § 60 odst. 1 autorského zákona.

V Plzni dne 01. března 2026

.....

Bc. Milan Janoch

V textu jsou použity názvy produktů, technologií, služeb, aplikací, společností apod., které mohou být ochrannými známkami nebo registrovanými ochrannými známkami příslušných vlastníků.

Abstrakt

xxx

Abstract

xxx

Klíčová slova

xxx • xxxx • xxx

Obsah

1	Úvod	4
2	Teoretická část	5
2.1	Agilní vývoj a specifikace požadavků	5
2.1.1	Uživatelské příběhy	7
2.1.2	Akceptační kritéria	8
2.2	Automatizované testování softwaru	9
2.2.1	E2E testování	11
2.2.2	Přehled frameworků pro webovou automatizaci	12
2.2.3	Výzvy automatizace v agilním prostředí	13
2.3	Velké jazykové modely ve vývoji softwaru	14
2.3.1	Parametry a technické aspekty LLM	15
2.3.2	Srovnání LLM modelů a pricing	17
2.3.3	Limity generování	18
2.4	Agentní systémy a autonomní testování	18
2.4.1	Definice a charakteristika autonomních agentů	18
2.4.2	Architektura ReAct: Spojení uvažování a jednání	19
2.4.3	Model Context Protocol (MCP)	20
2.5	Shrnutí	22
3	Návrh systému	23
3.1	Analýza struktury a sémantiky vstupních artefaktů	23
3.2	Specifikace požadavků na systém	26
3.2.1	Vymezení rozsahu generovaných testů	26
3.2.2	Funkční požadavky (FR) na systém	26
3.2.3	Nefunkční požadavky (NFR) na systém	27
3.3	Koncepční architektura systému	28
3.3.1	Doménový model aplikace	28
3.3.2	Distribuce a perzistence dat	29
3.3.3	Architektura systémových komponent	31

3.3.4	Architektura agenta a exekuční smyčka	33
3.4	Návrh uživatelského rozhraní	35
3.5	Shrnutí	37
4	Použité technologie	38
4.1	TypeScript	38
4.2	React	38
4.3	NestJS	39
4.4	Vercel AI SDK	40
4.5	Playwright a Model Context Protocol (MCP)	41
4.6	Shrnutí	41
5	Implementace	42
5.1	Struktura a organizace projektu	42
5.2	Implementace backendové části	43
5.2.1	Doménové moduly	43
5.2.2	Implementace doménového modelu a sdílených DTO . . .	45
5.2.3	Perzistence a konzistence dat	46
5.3	Implementace komunikace s LLM	48
5.3.1	Abstrakce poskytovatelů a Vercel AI SDK	48
5.3.2	Správa dostupných modelů	50
5.4	Implementace autonomního agenta	52
5.4.1	Integrace nástrojů přes Model Context Protocol	52
5.4.2	Řídící smyčka agenta a ochranné mechanismy	53
5.4.3	Návrh systémového promptu a řízení chování (Prompt Engineering)	56
5.4.4	Validační pipeline a samoopravná smyčka	57
5.5	Implementace klientské části	59
5.6	Shrnutí	59
6	Závěr	60
	Přehled zkratk	61
A	Specifikace promptů pro AI agenta	62
A.1	Systémový prompt	62
A.2	Prompt pro regeneraci testů	66
B	Uživatelská dokumentace	68
	Bibliografie	69

Seznam obrázků	74
Seznam tabulek	75
Seznam výpisů	76

V dnešním světě, kdy se software i díky nástupu umělé inteligence vyvíjí rychleji než kdy dříve, je zajištění kvality a spolehlivosti aplikací klíčovým faktorem pro úspěch. Testování hraje zásadní roli při odhalování chyb, zajištění důvěry uživatelů a zvyšování kvality softwaru. S nástupem velkých jazykových modelů (LLM) se objevují nové příležitosti pro automatizaci generování testů [1], což může výrazně zefektivnit proces testování a snížit náklady spojené s manuálním vytvářením testovacích scénářů. Hranice možností se stále posouvají nejen co se týče výkonností jednotlivých modelů (např. známých GPT, Claude) a společností přicházejících na trh, ale i co se týče přístupů k využití těchto modelů pro generování testů. I přes tento inovační potenciál je ale třeba myslet na rizika a výzvy spojené s rozšířeným využíváním LLM - a to nejen z pohledu softwarového (např. riziko halucinací, nedostatečná kvalita generovaných testů), ale i na cenovou a etickou stránku věci (např. náklady na využívání LLM, otázka ochrany dat a soukromí).

Cílem této diplomové práce je prozkoumat možnosti generování kódu pomocí velkých jazykových modelů v rámci automatizovaného testování a implementovat funkční prototyp, který má potenciál výrazně urychlit a zefektivnit proces testování softwaru pomocí end-to-end testů. Aby aplikace byla prakticky použitelná, bude zaměřená na generování testů na základě uživatelských příběhů, jež jsou běžně využívány v agilním vývoji softwaru [2] a poskytují přehledný a srozumitelný popis požadavků z pohledu uživatele. To umožní rozšířit možnosti psaní spustitelných testů i do oddělení jako je QA (Quality Assurance), které nemusí mít hluboké technické znalosti, ale jsou klíčové pro zajištění kvality softwaru. Důraz bude kladen na kvalitu vygenerovaných testů, jejich schopnost odhalovat chyby a přínos pro zefektivnění procesu testování.

Teoretická část

2

V této kapitole bude představen teoretický rámec nezbytný pro pochopení problematiky autonomního generování testů. Z důvodu potřeby transformace obchodních požadavků do funkčního kódu bude nejprve popsáno, jakým způsobem probíhá specifikace v agilním prostředí pomocí uživatelských příběhů a akceptačních kritérií. Následně budou rozebrány moderní přístupy k testování softwaru, existující frameworky pro webovou automatizaci a výzvy spojené s testováním v agilním prostředí.

Důležitou částí kapitoly je rozbor velkých jazykových modelů (LLM), které v současnosti slouží jako nástroj pro interpretaci požadavků a generování kódu. Budou vysvětleny jejich základní principy, technické parametry a limity spojené s psaním testovacích skriptů. V závěru kapitoly pak budou představeny autonomní agentní systémy, které propojují svět testování a umělé inteligence skrze pokročilé architektury a komunikační protokoly. Tento ucelený přehled metodik a technologií poslouží jako základ pro následný praktický návrh vlastního řešení.

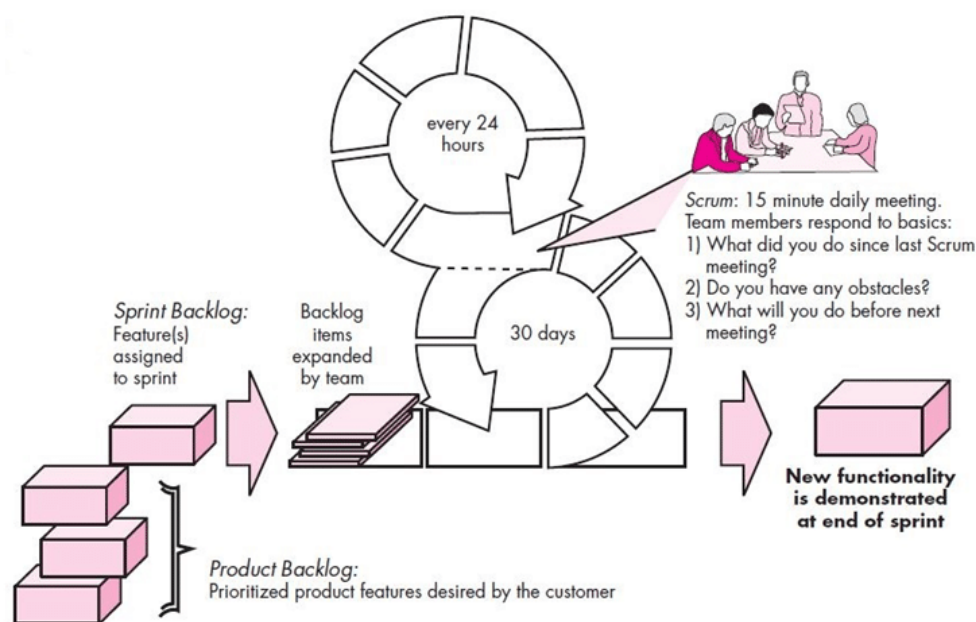
2.1 Agilní vývoj a specifikace požadavků

IT je jedním z nejrychleji se rozvíjejících odvětví, a to nejen z pohledu technologií, ale i přístupu k vývoji softwaru. Jak [3] uvádí, tento aspekt vedl k vývoji metodik (např. agilní metodiky či vodopádový model), které navrhují řízení v závislosti na vlastnostech produktu, organizačních strukturách či dynamickém prostředí. Tradiční konzervativnější vodopádový model (populární např. ve stavebnictví) se zaměřuje na důkladné plánování [4] a znalost požadavků před zahájením vývoje [5]. V rychle se měnícím prostředí je ale tento přístup často neefektivní, protože požadavky ze strany uživatelů se mohou rychle měnit a není možné je znát všechny předem. Tento problém řeší agilní metodiky, které kladou důraz na flexibilitu - díky tomu, že fungují na tzv. iterativním principu (software se vyvíjí v krátkých cyklech často přezdívaných jako sprinty), umožňují průběžné přizpůsobování se měnícím se požadavkům a rychlejší dodávání funkčních verzí softwaru [5].

Nejznámějším a současně nejrozšířenějším přístupem k agilnímu vývoji je Scrum. Scrum tým se zpravidla skládá z následujících rolí [4, 6, 7, 8]:

- **Vývojáři (Developers)** - jsou zodpovědní za vývoj softwaru a implementaci požadovaných funkcionalit (features).
- **Scrum Master** - zajišťuje dodržování Scrum procesů, podporuje tým při práci a pomáhá odstraňovat překážky, které by mohly bránit efektivnímu vývoji.
- **Product Owner** - je zodpovědný za správu produktového backlogu, komunikaci s vývojovým týmem a zainteresovanými stranami (stakeholdery) a za zajištění toho, aby vývoj směřoval k naplnění potřeb zákazníka.

Hlavní náplní práce Product Ownera je správa backlogu a příprava uživatelských příběhů (často známý pod rozšířenějším anglickým pojmem *user stories*), které specifikují požadované funkcionality z pohledu uživatele [8].



Obrázek 2.1: Vizualizace Scrum procesu a jeho klíčových fází [9].

Vzájemná interakce těchto rolí a posloupnost jednotlivých aktivit je znázorněna na obrázku 2.1. Celý proces probíhá v iterativních cyklech (tzv. sprintech), které zpravidla trvají 2-4 týdny (na zmiňovaném schématu je pro ilustraci uvedena historicky tradiční délka 30 dnů). Během nich tým postupuje od výběru prioritních požadavků zákazníka z produktového backlogu (*Product Backlog*), přes vytvoření

menšího backlogu pro daný sprint (*Sprint Backlog*), až po samotnou implementaci, která je prokládána každodenními krátkými (15-minutovými) schůzkami. Klíčovým výstupem každého sprintu je funkční přírůstek softwaru a zpětná vazba od zákazníka. A právě uživatelské příběhy tvoří základní pilíř pro specifikování těchto přírůstků.

2.1.1 Uživatelské příběhy

User story je nejpoužívanějším artefaktem v agilním vývoji [10] - jedná se o krátký, semi-strukturovaný popis požadavků napsán v přirozeném jazyce. Na ukázce 2.1 je uvedena standardní šablona, podle níž se uživatelské příběhy píší [10, 11, 12].

As a <type of user / persona>
I want <to perform some action / goal>
So that <some value / benefit is achieved>

Zdrojový kód 2.1: Standardní šablona uživatelského příběhu.

Každá user story by měla být napsána z pohledu konkrétního uživatele (*persona*), s jasně stanoveným cílem (*goal*) a dosaženou hodnotou (*value*). Hodnota je dosažena pouze tehdy, pokud byl plánovaný cíl úspěšně splněn [12].

Kvalita napsaných user stories je klíčová nejen pro porozumění vývojářů při implementaci, ale také pro následné testování a ověření, že plánovaný cíl byl skutečně dosažen. Jednou z metod, která pomáhá jasně definovat kritéria pro úspěšné splnění user story, je INVEST [13]. V tabulce 2.1 jsou uvedeny jednotlivé principy INVEST, které by měly být dodržovány pro psaní kvalitních uživatelských příběhů.

Princip	Popis
Independent	User story by měla být co nejvíce nezávislá na ostatních stories.
Negotiable	Požadavky by měly být otevřené pro diskusi a případné úpravy.
Valuable	Splnění user story musí přinést jasnou hodnotu koncovému uživateli.
Estimable	Musí být možné odhadnout náročnost implementace (např. čas nebo potřebné zdroje).
Small	User story by měla být dostatečně malá, aby bylo možné ji dokončit během jednoho sprintu.
Testable	Musí obsahovat definovaná akceptační kritéria, podle kterých lze ověřit splnění požadavku.

Tabulka 2.1: INVEST principy pro tvorbu kvalitních user stories [14].

2.1.2 Akceptační kritéria

Zatímco user story charakterizují potřeby zákazníka, akceptační kritéria (často označována zkratkou ACC z anglického Acceptance Criteria) jasně popisují Definition of Done (zkráceně DoD) pro konkrétní user story [15] co je nutno dodržet a udělat, aby byla user story považována za splněnou. Týkají se zejména funkcionality (tlačítko dělá to co má), kvalitativních charakteristik (systém reaguje do X milisekund) a obchodní logiky [16].

Akceptační kritéria mohou být zapsána různou formou - ať už prostý seznam, kde každý bod představuje právě jedno kritérium, nebo formou scénářů, které jsou popisovány pomocí jazyka Gherkin. Tento způsob testování (kdy jsou nejprve napsány testovací scénáře v přirozeném jazyce a následně se implementují samotné testy) se nazývá Behavior-Driven Development (BDD) [17].

Feature: User Login

Scenario: Successful login with valid credentials

Given the user is on the "/login" page

When the user enters "user@example.com" into the email field

And the user enters "password123" into the password field

And the user clicks the "Login" button

Then the user should be redirected to the "/dashboard" page

And a welcome message "Welcome back" should be displayed

Zdrojový kód 2.2: Ukázka testovacího scénáře zapsaného pomocí klíčových slov jazyka Gherkin.

Hlavní motivací pro využití Gherkin scénářů je vysoká abstrakce - scénáře jsou psány v přirozeném jazyce (jak je vidět na ukázce 2.2), což umožňuje jednodušší komunikaci mezi technicky zdatnými i netechnickými členy týmu a zároveň poskytují jasný popis požadavků a očekávaného chování [18].

Pro prostý seznam akceptačních je důležité, aby používal jasný a konzistentní jazyk a zaměřoval se hlavně na testovatelnost, nikoliv na formát zápisu [16]. Ukázka takového seznamu je uvedena na ukázce 2.3 - každý bod by měl být jasně identifikovatelný a obsahuje jedno kritérium, které lze ověřit.

Acceptance Criteria for US_01: User Login

- **ac_01 (Success):** System grants access to the dashboard when valid credentials (email and password) are provided.
- **ac_02 (Success):** Navigation to the login page is possible via the "Sign In" button on the landing page.
- **ac_03 (Failure):** System displays an error message "Invalid credentials" if the password does not match the stored hash.
- **ac_04 (Failure):** Account is temporarily locked after five consecutive failed login attempts.
- **ac_05 (Robustness):** Input fields prevent the submission of SQL injection patterns or excessively long strings.

Zdrojový kód 2.3: Ukázka akceptačních kritérií zapsaných formou strukturovaného seznamu.

V praxi bývají akceptační kritéria (spolu s uživatelskými příběhy) nejčastěji obsažena v ALM nástrojích (Application Lifecycle Management) jako jsou Jira, Azure DevOps, Rally či GitHub Projects. Tyto nástroje nevyžadují specifický formát zápisu, ale umožňují flexibilitu - ať už jde o prostý seznam, Gherkin scénáře nebo kombinaci obojího. Např. studie *From bugs to benefits: Improving user stories by leveraging crowd knowledge with cruise-ac* [19] zaměřená na generování akceptačních testů vycházela z běžných user stories a jejich textových popisů, z něhož následně generovala Gherkin scénáře, z nichž až poté byly implementovány automatizované testy.

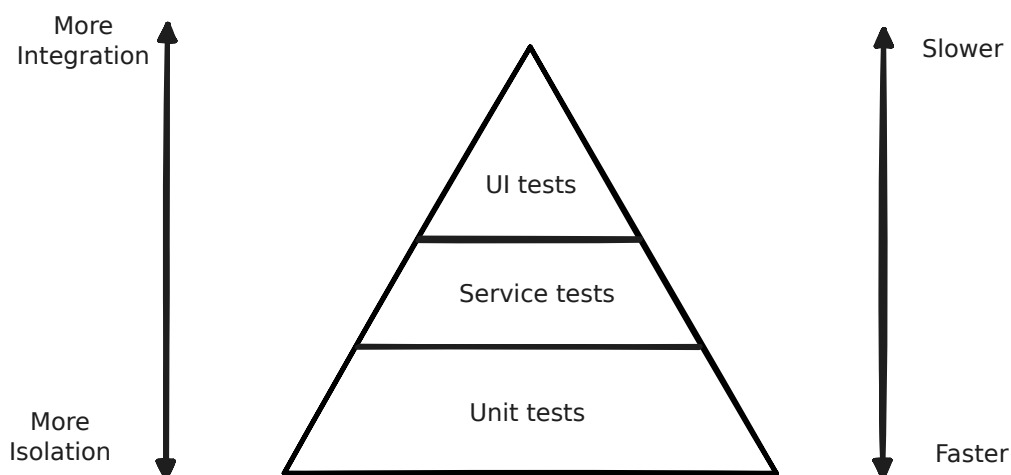
2.2 Automatizované testování softwaru

Softwarové testování lze definovat jako hledání chyb či chybějících požadavků v softwaru [20]. Dá se na něj nahlížet i z pohledu snižování rizika, což vede v posílení důvěry uživatelů a zlepšení kvality softwaru. Jak také *Software Testing Techniques: A Literature Review* [20] uvádí, testování je důležitou součástí QA - čím dříve jsou chyby odhaleny, tím levnější je jejich oprava. Testování lze rozdělit do několika tříd, jež jsou znázorněny v tabulce 2.2, která vychází z knihy [21].

Název testu	Popis
Testování podle scénářů	Tester dle předpřipravených scénářů manuálně prochází aplikaci a kontroluje očekávané chování jednotlivých testovacích kroků.
Průzkumné testování	Tester bez předem známých scénářů prozkoumává aplikaci a hledá chyby.
Regresní testy	Ověřují, zda nově přidáné featury nerozbily existující funkce.
Jednotkové (unit) testy	Automatizované skripty, které izolovaně ověřují funkčnost nejmenších částí zdrojového kódu (funkce, metody).
Beta testy	Poskytnutí aplikace omezené skupině uživatelů.
Smoke testy	Rychlé testy ověřující sestavení aplikace a její stabilitu.
Konfirmační testy	Potvrzení, že aplikace po odstranění defektu funguje správně.

Tabulka 2.2: Druhy testů dle [21].

Na obrázku 2.2 je uvedena pyramida testování, která znázorňuje důležitost jednotlivých druhů testů v rámci automatizace testování [22].



Obrázek 2.2: Pyramida testování Mika Cohna.

Jednotkové testy jsou nejvíce izolované, píšou se rychleji, a tudíž jsou i levnější. Naproti tomu UI testy (User Interface - testy uživatelského rozhraní - např. end-to-end) vyžadují větší integraci a jejich vývoj je zpravidla dražší (testy se píšou déle a déle běží). Přestože jsou end-to-end testy dražší, představují nedílnou součást celé automatizace - bez těchto testů není zaručeno, že komponenty/moduly systému spolu správně komunikují a dávají tak koncovému uživateli očekávaný stav.

Při psaní testů se objevují speciální případy testů - a to testy chybně napsané či nestabilní testy. Jedná se o následující termíny [21]:

- **Falešně pozitivní (False Positive)** - Test, který selže, přestože testovaná funkcionality funguje správně.
- **Falešně negativní (False Negative)** - Test, který projde, i když testovaná funkcionality nefunguje správně.
- **Křehké (Fragile)** - Test, který selže kvůli změně v implementaci, přestože testovaná funkcionality zůstala nezměněna (např. změna názvu HTML elementu).
- **Nestabilní (Flaky)** - Test, který se chová nedeterministicky - může selhat nebo projít bez jakýchkoliv změn v kódu, což ztěžuje identifikaci skutečných problémů (např. problémy se síťovou komunikací).

Těmto problémům se při psaní testů snaží vývojáři předcházet - a to např. důrazem na determinismus testů, kontrolou správnosti testovaného chování na Code Review nebo využíváním nástrojů pro správu testů, které pomáhají identifikovat a řešit nestabilní testy.

2.2.1 E2E testování

End-to-end testování (zkráceně E2E) je typ testování, který ověřuje kompletní funkčnost systému z pohledu koncového uživatele. Využívá se přístupu tzv. metody **black-box** (černé skříňky), kdy tester nemá přístup k implementaci či struktuře systému, ale testuje systém jako jeden celek [21, 23]. V případě webových aplikací je možné nahlížet na způsoby tvorby end-to-end testů dvěma způsoby [23]:

- **Capture-Replay** - Tester nejdříve manuálně prochází aplikaci, zaznamenává své akce a následně se tyto akce převedou na automatizovaný skript.
- **Programmable Web Testing** - Testy se rovnou píšou jako zdrojový kód, který využívá frameworky pro webovou automatizaci.

Hlavní výhodou přístupu **Capture-Replay** je rychlost - testy lze vytvořit velmi rychle, nevýhodou je ale nízká kvalita a nestabilita testů, které jsou často náchylné k falešně negativním výsledkům (např. když se HTML elementy změní, program funguje stále správně, ale test selže).

Obě tyto perspektivy potřebují lokalizovat a interagovat s HTML elementy. Interakce s HTML elementy lze rozdělit do tří kategorií [23]:

- **Coordinate-based** - Využívání [X,Y] souřadnic pro interakci s elementy.
- **DOM-based** - Využití Document Object Modelu (DOM)/HTML struktury.

- **Vizuální** - Využití rozpoznání obrazu pro určení elementů na obrazovce.

Zatímco první kategorie je označována jako zastaralá [23], DOM-based přístup je náchylný na drobné změny v HTML struktuře, které mohou způsobit selhání testů a vyžadují častou údržbu. Vizuální přístup sice dokáže testovat to, co uživatel skutečně vidí, ale je náchylný na změny v rozložení stránky a režie spojená s rozpoznáváním obrazu může být vysoká. Právě snaha o řešení těchto problémů vedla k zrodu mnoha testovacích frameworků pro webovou automatizaci, které se snaží zjednodušit a zefektivnit tvorbu end-to-end testů.

2.2.2 Přehled frameworků pro webovou automatizaci

V oblasti webové automatizace existuje velké množství frameworků, které se navzájem liší svými vlastnostmi, podporovanými jazyky a přístupy k testování. Zde je uveden přehled těch nejpopulárnějších a nejčastěji používaných,

Selenium (WebDriver)

Jeden z nejrozšířenějších a nejstarších nástrojů. Podporuje širokou škálu jazyků (Java, Python, C#, JavaScript) a prohlížečů prostřednictvím WebDriverů [24]. Díky rozšířené komunitě, množství dostupných zdrojů a existujících knihoven je Selenium stále velmi populární volbou. Nevýhodou je složitější prvotní konfigurace a manuální správa testů, která může (při nesprávném použití) vést k nestabilním testům.

Robot Framework

Open-source framework založený na Pythonu, který využívá klíčová slova pro psaní testů (tzv. Keyword-Driven Testing). Architektura využívá knihovny SeleniumLibrary pro webovou automatizaci [25]. Silnou stránkou je jednoduché psaní testů - testy jsou psány čitelně v přirozeném jazyce, díky čemuž jim porozumí i netechničtí členové týmu. Se zvyšující se komplexností webových aplikací a testů může být nevýhodou nižší flexibilita a nutnost používat externí knihovny pro pokročilé funkce.

Playwright

Moderní testovací framework vyvinutý společností Microsoft. Podporuje více jazyků (JavaScript/TypeScript, Python, Java, .NET) a nabízí robustní API (Application Programming Interface) pro interakci s webovými stránkami [26]. Výhodou je rychlost testů a podpora pro více prohlížečů (Firefox, WebKit, Chromium) bez nutnosti externích driverů. Klíčovou funkcí je také Auto-waiting, který pomáhá eliminovat

křehkost testů. Nevýhodou je menší komunita a méně dostupných zdrojů, jelikož se jedná o relativně nový nástroj.

Cypress

Nástroj zaměřený na testování moderních webových aplikací, který podporuje pouze JavaScript a TypeScript. Běží přímo v prohlížeči [27], což urychluje vývoj testů a jejich případné debuggování. Disponuje také funkcí *Time Travel*, která umožňuje procházet test krok po kroku a vidět stav aplikace v každém kroku. Nevýhodou je omezená podpora napříč jazyky, prohlížeči a testování vícero tabů.

2.2.3 Výzvy automatizace v agilním prostředí

Studie *A conceptual model supporting decision-making for test automation in Agile-based Software Development* [28] identifikovala několik klíčových výzev, které se vyskytují při implementaci automatizace testů v agilním prostředí.

Údržba a technický dluh

Jak vyplývá ze zmíněné studie, tento proces je časově náročný a náchylný k chybám. S rostoucím počtem uživatelských příběhů se manuální tvorba testů (testovacích případů) stává bottleneckem, což může vést k následnému nárůstu technického dluhu, kdy se testy stávají zastaralými a všechny scénáře nemusí být pokryty, což zvyšuje riziko chyb.

Regresní testování

Jelikož agilní vývoj vyžaduje rychlé a časté změny, je klíčové zajistit, aby po každé z nich bylo co nejdříve ověřeno na regresních testech, že nedošlo k narušení stávající funkcionality. Tento problém je možno částečně řešit automatickým generováním testů přímo z požadavků, aby se minimalizoval čas mezi implementací a testováním.

Rozdíl mezi specifikací a kódem

Tato výzva souvisí s jasností požadavků na produkt. Jelikož jsou požadavky napsány v přirozeném jazyce, může docházet k nedorozuměním či nejasnostem, které mohou vést k nesprávné implementaci či chybám. To má poté za následek:

- Nestabilní testovací orákulum - pokud mechanismus pro ověření výsledků není stabilní a předvídatelný, může docházet k velkému množství falešně pozitivních nálezů či křehkým testům.

- Náklady na opravu - opakovaná reimplementace funkcionalit vynucuje souběžnou údržbu testů, což výrazně snižuje návratnost (ROI) investic.
- Nízká testovatelnost - zaměření na testovatelnost od prvotních fází vývoje výrazně zvyšuje efektivitu automatizace.

2.3 Velké jazykové modely ve vývoji softwaru

Velké jazykové modely jsou rozsáhlé, předtrénované, statistické modely založené na neuronových sítích. Jedná se zejména o modely založené na architektuře Transformer, které obsahují miliardy parametrů a jsou trénovány na masivním množství textových dat. Díky tomuto objemu parametrů jsou schopni zachytit komplexní vzory a vztahy v jazyce (na základě statistické analýzy) a generovat text [29]. Navíc vykazují i tzv. **emergent abilities** - schopnosti jako je schopnost učit se v kontextu, plnit komplexní instrukce či řešit více krokové úlohy.

Evoluce jazykových modelů prošla 4 hlavními generacemi [30]:

- **Statistické jazykové modely (SLM)** - Vznik v 90. letech 20. století, založené na pravděpodobnostních modelech a n-gramových přístupech.
- **Neuronové jazykové modely (NLM)** - Slova mapovaná na vektory (embeddings), k predikci dalších slov využívány rekurentní neuronové sítě (RNN) a později LSTM (Long Short-Term Memory).
- **Předtrénované jazykové modely (PLM)** - Předtrénované na obrovském množství neoznačeného textu a využití fine-tuningu pro zaměření na konkrétní úlohy.
- **Velké jazykové modely (LLM)** - Současné modely s miliardami parametrů vycházející z PLM, které díky masivnímu nárůstu dat a výpočetnímu výkonu dokáží řešit mnoho úloh bez nutnosti dalšího doladování (emergent abilities).

Základem architektury Transformer, která byla představena v roce 2017 [31] a je převratná pro vývoj LLM, je tzv. mechanismus **Self-Attention** - ten umožňuje modelovat vliv každého slova v kontextu věty na ostatní slova, což pomáhá efektivněji zachytit a pochopit kontextové vazby.

V rámci architektury Transformer se využívají dva hlavní typy bloků - **Encoder** a **Decoder**. Zatímco Encoder bloky se zaměřují na zpracování vstupního textu a jejich pochopení, Decoder bloky jsou zaměřené na generování výstupního textu na základě těchto reprezentací [30]. Na modely se pak nahlíží tak, jakou část této architektury využívají:

- **Encoder-only** - Modely využívající pouze encoder část architektury. Jsou vhodné zejména pro úlohy zaměřené na porozumění textu, například klasifikaci nebo analýzu sentimentu. Typickým příkladem je BERT.
- **Decoder-only** - Modely založené pouze na decoder části, optimalizované pro generování textu. Do této kategorie patří například rodina modelů GPT.
- **Encoder-Decoder** - Modely kombinující obě části architektury. Využívají se zejména pro úlohy transformace textu, jako je strojový překlad, sumarizace nebo otázky a odpovědi. Příkladem je model T5.

Je nutné podotknout, že i přes své schopnosti a potenciál jsou LLM stále jen nástrojem založené na predikci - nejsou schopné skutečnému porozumění, a proto může docházet k jevu zvanému halucinace - okamžik, kdy model generuje věcně nesprávný či nesmyslný text, který ale může působit věrohodně.

2.3.1 Parametry a technické aspekty LLM

Tokenizace a zpracování dat

Tokenizace je proces převedení vstupního textu na menší jednotky (tzv. tokeny), které mohou reprezentovat celá slova, části slov či jednotlivé znaky. To umožňuje modelům zpracovávat i neznámé výrazy pomocí algoritmů jako je Byte Pair Encoding (BPE) nebo SentencePiece [32].

Tyto tokeny se dále dělí na další kategorie uvedené v tabulce 2.3, které jsou i základní metrikou pro pricing jednotlivých poskytovatelů (providerů) v rámci používání jejich modelů.

Token	Popis	Příklad	Cena
Input	Data zasílaná modelu ke zpracování.	Uživatelský dotaz.	Nižší
Output	Sekvence tokenů generovaná modelem jako odpověď.	Odpověď modelu.	Vyšší
Reasoning	Externalizovaný výpočetní stav předávaný mezi cykly generování.	Mezikroky řešení.	Dle implementace
Cached	Data v mezipaměti pro opakované využití.	Části dlouhého dokumentu.	Nejnižší

Tabulka 2.3: Přehled běžně používaných druhů tokenů v LLM.

Přestože reasoning tokeny mohou připomínat lidské uvažování, model po dalším cyklu tuto znalost ztrácí - tyto tokeny mu tak slouží jako State over Tokens, kam si odkládá mezivýsledky, aby věděl, jak při dalším kroku pokračovat [33].

Context window

Kontextové okno definuje maximální počet tokenů, které je model schopen zpracovat v rámci jednoho výpočetního cyklu. Zatímco starší generace modelů operovaly v řádu nižších tisíců tokenů, současné modely již zvládají pracovat v řádu nižších milionů. Průkopníkem v této oblasti se stala společnost Google, která jako první u svých modelů řady Gemini ¹ představila kapacitu přesahující jeden milion tokenů. Podobné limity nyní postupně adoptují i další poskytovatelé, jako například Anthropic u modelů Claude v rámci platformy Bedrock ².

Temperature

Parametr využívaný modely k určení míry náhodnosti a diverzity generovaného výstupu [34]. Čím nižší je hodnota `temperature`, tím konzervativnější a determinističtější je výstup modelu. S vyššími hodnotami stoupá riziko halucinací, ale model nabízí vyšší míru kreativity. Pro úlohy vyžadující přesnost a konzistenci se doporučuje nastavit hodnotu `temperature` na 0. Některé modely (např. v rámci Azure OpenAI) tento parametr neumožňují konfigurovat, protože je pevně definován na úrovni modelu ³.

Tool Calling

Tool Calling (volání nástrojů) je schopnost modelu autonomně využívat externí aplikace či služby (jako jsou firemní API, vyhledávače...), čímž překonávají svá omezení jako jsou např. zastaralé informace. Pro správné použití je nutné zajistit, aby model mohl správně identifikovat záměr uživatele, vybrat vhodný nástroj a správně nastavit vstupní parametry [35]. Tuto schopnost lze modelu naučit buď pomocí fine-tuningu na ukázkových interakcích, nebo technikou `in-context-learningu`, kdy se do promptu, který se zasílá modelu, vloží definice jednotlivých nástrojů, aby věděl, jak je využívat. Tato vlastnost není podporována u všech modelů, a proto je třeba předem zkontrolovat dokumentaci, zda je pro daný model Tool Calling dostupný.

Prompt Caching

Prompt Caching je inovativní technika, jež výrazně zrychluje inferenci modelu a snižuje výpočetní nároky tím, že recykluje interní výpočty modelu [36]. Vychází z toho, že uživatelské dotazy obsahují často stejné bloky zpráv (např. systémový prompt, specifikace nástrojů, přiložené dokumenty...), které se mezi dotazy nemění. Namísto přepočítávání těchto bloků si model ukládá jejich výpočty do mezipaměti

¹Dokumentace Google AI

²Dokumentace AWS Bedrock (Anthropic)

³Dokumentace Microsoft k Azure OpenAI

a při dalším dotazu, který obsahuje identické bloky, si je pouze načte z mezipaměti a spojí, čímž se zkracuje doba zpracování dotazu a to bez jakéhokoliv dopadu na kvalitu finálního výstupu. Tato technika je podporována zejména u novějších modelů a je nutné zkontrolovat dokumentaci, zda je pro daný model dostupná.

2.3.2 Srovnání LLM modelů a pricing

Výběr konkrétního modelu závisí na mnoha faktorech - ať už se jedná o výkon, ale i rovnováhu mezi cenou, rychlostí a dostupností funkcí (Tool Calling, Temperature, podporovaný vstup - text, obrázky). V tabulce 2.4 jsou uvedeny některé z nejnovějších modelů, které vyšly od druhé poloviny roku 2025.

Provider	Model	In (\$/1M)	Out (\$/1M)	Context
Anthropic	Claude 4.6 Sonnet	3.00	15.00	1M
	Claude 4.6 Opus	5.00	25.00	1M
OpenAI	GPT-5 (Standard)	1.25	10.00	400k
	GPT-5.3 Codex	1.75	14.00	400k
Google AI	Gemini 3.1 Pro	2.00/4.00	12.00/18.00	1M
	Gemini 3.1 Flash-Lite	0.25	1.50	1M
Ostatní	DeepSeek-V3.2	0.28	0.42	128k
	Llama 4 Scout	0.18	0.59	1M
	Moonshot Kimi K2	0.60	2.50	262k

Tabulka 2.4: Vybrané modely vydané od H2 2025 a jejich cenová politika.

Veškeré cenové údaje jsou převzaty z oficiálních dokumentací jednotlivých poskytovatelů.⁴ Je důležité poznamenat, že každý poskytovatel uplatňuje odlišnou cenovou strategii. Například, jak je patrné z tabulky 2.4, Google AI rozlišuje u některých modelů dvoustupňový pricing - sazba za milion tokenů se automaticky navyšuje u dotazů, jejichž celkový kontext přesáhne hranici 200 000 tokenů. A ještě větší rozdíly jsou u přístupu ke cachování - zatímco např. OpenAI sází na plnou automatizaci a slevy pro krátkodobě identické začátky promptů, Anthropic a Google vyžadují explicitní definici ukládaných bloků, což umožňuje efektivnější a dlouhodobější uchování rozsáhlých dat.

Modely, které jsou veřejnosti dostupné (včetně jejich metadat o nástrojích, kontextových oknech apod.), je možné najít na stránkách models.dev, která tak slouží jako centrální repozitář, který usnadňuje vývojářům výběr a porovnání modelů pro jejich konkrétní potřeby.

⁴Podrobné specifikace dostupné v dokumentacích: Anthropic, Google AI, OpenAI, DeepSeek, Together AI a Moonshot.

2.3.3 Limity generování

Ačkoliv LLM prokazují schopnost psát zdrojové kódy, při generování testů na GUI (Graphical User Interface) narážejí na omezení spojená s absencí interaktivity s aktuálním prostředím. Hlavními výzvami jsou:

- **Halucinace** - Jazykové modely si mohou při nedostatku kontextu vymýšlet neexistující elementy/selektory. S tímto chováním se setkal Horínek ve své diplomové práci [37], kdy se modely snažily použít atribut `aria`, který ovšem daným frameworkem nebyl vůbec podporován.
- **Statický kontext** - Model při generování testu nevidí reálný stav DOMu v čase běhu a nedokáže reagovat na dynamické změny na stránce. Tento problém demonstruje Horínek [37] na případu, kdy model generoval výběr prvku z rozbalovací nabídky, ovšem tyto hodnoty se při každé inicializaci databáze měnily a pevně vygenerované selektory tak v době běhu selhávaly.
- **Absence zpětné vazby a zotavení z chyb** - Bez možnosti interakce model nemůže v průběhu testu ověřit, zda jeho předchozí kroky byly úspěšné. Na nutnost řešit tento problém při návrhu nástrojů pro generování GUI testů poukazuje ve své práci Holický [38]. Pokud je test vygenerován čistě staticky, se-bemenší nepřesnost nebo neočekávané chování uživatelského rozhraní vede k selhání celého skriptu. Bez implementace dodatečné zpětné vazby (např. předání chybové hlášky zpět modelu) se systém z takové chyby nedokáže zotavit a test zůstává nevalidní.

Tyto tři body ukazují, že generovat kód staticky pro moderní proměnlivé aplikace nemusí stačit. Pro překonání těchto limitů je potřeba opustit paradigma jednorázového promptu a přijít s řešením, které dokáže s aplikací aktivně interagovat, pozorovat výsledky akcí a následně je verifikovat. Tímto směrem se ubírají tzv. **agentní systémy** - systémy, které vidí do DOMu a dokážou dynamicky reagovat.

2.4 Agentní systémy a autonomní testování

2.4.1 Definice a charakteristika autonomních agentů

S narůstající komplexností úloh, které jsou na modely umělé inteligence kladeny, se objevuje potřeba systémů k migraci od tradičního jednorázového generování k interaktivním a adaptivním systémům, které dokáží reagovat na dynamické prostředí. Zatímco standardní neagentní přístup (tzv. *non-agentic workflow*) funguje na principu, který generuje výstup na základě jednorázového uživatelského dotazu

a čeká na další lidský pokyn [39], agentní systémy představují zásadní změnu paradigmatu. V agentním pojetí se LLM stává jakýmsi “mozkem“, jenž proaktivně řídí provádění úkolů, samostatně se rozhoduje o dalším postupu a v případě potřeby iterativně reviduje své kroky bez nutnosti neustálých zásahů člověka [39, 40, 41].

Ve své základní podobě se skládá ze tří klíčových komponent [41]:

- **Model** - Konkrétní velký jazykový model, který řídí uvažování, rozhodování a celkovou exekuci workflow agenta.
- **Nástroje (Tools)** - Externí funkce nebo API, jenž agent využívá k interakci s vnějším prostředím.
- **Instrukce (Instructions)** - Explicitní pravidla definující agentovo chování - zmenšují prostor pro nejednoznačnosti což vede k efektivnějšímu chování a minimalizaci halucinací.

Pro plné fungování v rámci komplexnějších architektur je často zmiňují i další klíčové aspekty, které pomáhají agentům řešit náročnější úlohy. Mezi ně patří paměť (**Memory**), která umožňuje agentovi uchovávat a využívat informace z předchozích interakcí, a dále plánování (**Planning**), jež umožňuje agentovi dekomponovat složitý problém na sadu menších proveditelných kroků [42].

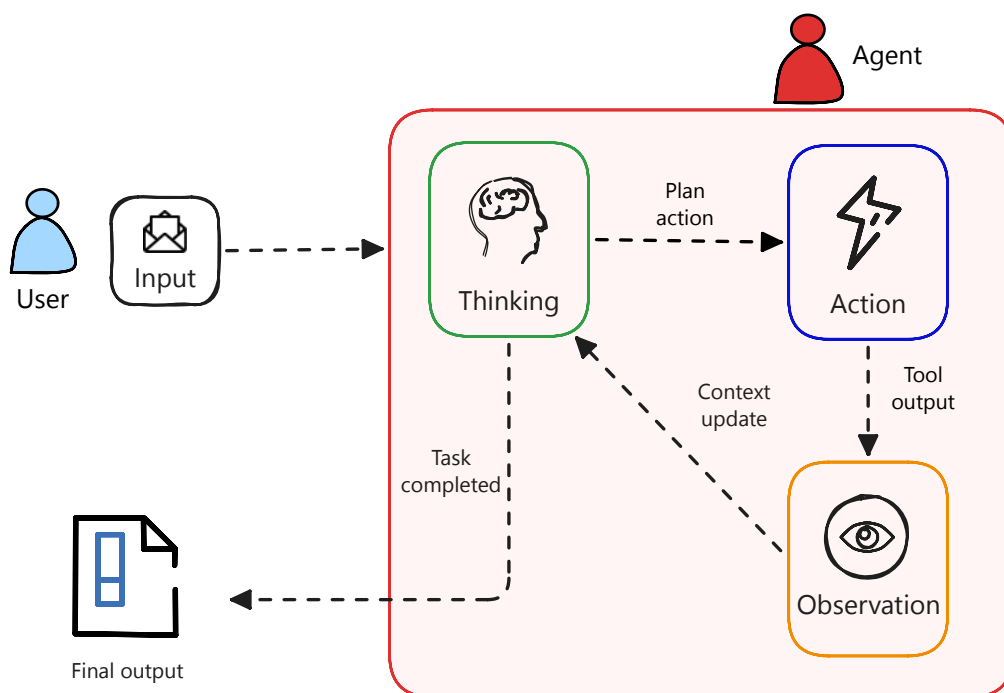
2.4.2 Architektura ReAct: Spojení uvažování a jednání

Vzor ReAct (**Reasoning and Acting**) je klíčový rámec pro pochopení toho, jak autonomní agenti “přemýšlejí“ a jednají [40]. Tento vzor umožňuje jazykovému modelu krok za krokem uvažovat o problému a konkrétními příkazy spouštět nástroje. Díky tomu model (místo klasického přístupu, kdy se vygeneruje celý výstup najednou) běží v cyklu, jenž mu umožní iterativně vytvářet, monitorovat a revidovat své kroky.

Jádrem tohoto vzoru je opakující se smyčka, která se skládá ze tří fází [43]:

- **Thought (Myšlenka)** - Interní monolog, v němž model analyzuje aktuální stav. Na základě svých předchozích instrukcí a aktuálního kontextu hodnotí, co vidí, jaký by měl být další krok a jaké nástroje by měl k jeho dosažení použít.
- **Action (Akce)** - Konkrétní volání toolu, který model na základě předchozí úvahy zvolil.
- **Observation (Pozorování)** - Získání zpětné vazby z prostředí po provedení konkrétní akce - tou může být např. pozměněný stav DOMu, nová URL adresa či chybová hláška.

Celý tento proces je zobrazen na obrázku 2.3. Ten začíná přijetím vstupu od uživatele a pokračuje do agentovy iterativní smyčky, kde se střídají fáze uvažování, jednání a následného pozorování. Celý proces končí v okamžiku, kdy agent v rámci fáze myšlení dospěje k závěru, že úloha je splněna a vygeneruje výsledný výstup pro uživatele.



Obrázek 2.3: Vzor ReAct: Spojení uvažování a jednání pro autonomní agenty.

Pro QA a automatizované testy představuje tento vzor zásadní posun - díky explicitním krokům je celý proces generování plně transparentní, interpretovatelný a auditovatelný. Uživatel přesně vidí úvahu, která agenta vedla k výběru určitého lokátoru, což výrazně usnadňuje ladění a zvyšuje tak spolehlivost celého systému.

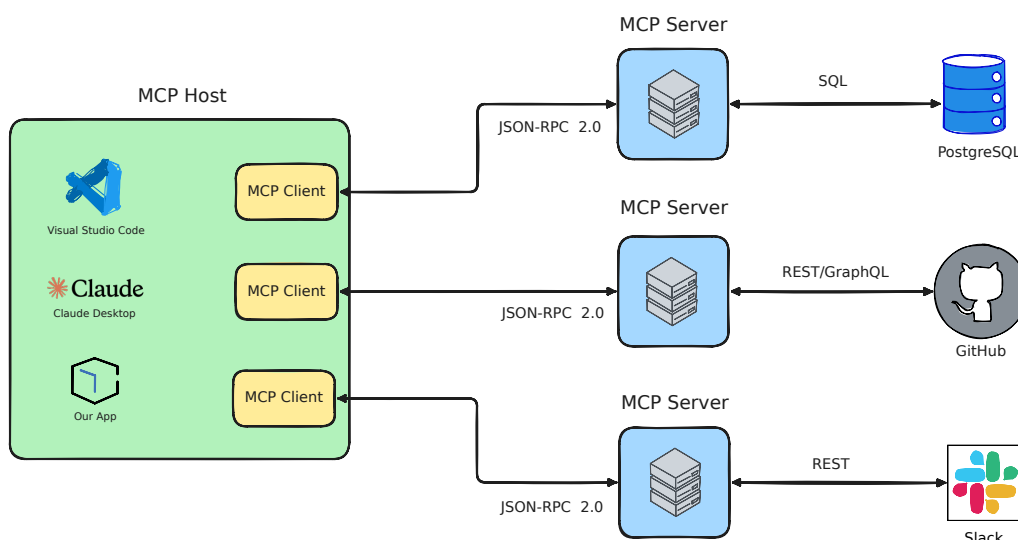
2.4.3 Model Context Protocol (MCP)

Model Context Protocol (MCP), představený firmou Anthropic ke konci roku 2024, je otevřený standard pro interoperabilitu, jehož cílem je sjednotit a zjednodušit způsob, jakým se modely umělé inteligence propojují s externími nástroji, systémy a daty.⁵ MCP, jenž si vysloužil přezdívku “USB-C for AI applications“, poskytuje jednotné a standardní rozhraní pro komunikaci - je postaven na robustním modelu komunikace Client-Host-Server s využitím formátu JSON-RPC 2.0 [42]:

⁵Oficiální dokumentace MCP

- **Client** - Protokolová komponenta zabudovaná do hostitelské aplikace, která udržuje spojení s jedním konkrétním MCP serverem.
- **Host** - Aplikace, která navazuje komunikaci a interaguje s uživateli.
- **Server** - Samostatný nezávislý proces (běžící lokálně nebo vzdáleně), který zpřístupňuje konkrétní funkce (tools) - slouží jako obálky kolem externích systémů (API, databáze, CLI nástroje...).

Obrázek 2.4 zobrazuje ukázkový ekosystém MCP, v němž hostitelské aplikace - od vývojových prostředí (IDE) jako je Visual Studio Code až po vlastní specializované platformy - komunikují se servery pomocí jednotného rozhraní. Díky této architektuře servery fungují jako univerzální adaptéry, překládají požadavky modelu do specifických protokolů (SQL, REST API) pro externí služby typu GitHub či Slack, což radikálně zvyšuje modularitu a zjednodušuje integraci nových funkcí do agentních systémů.



Obrázek 2.4: Ukázková architektura Client-Host-Server v rámci Model Context Protocolu.

Zásadní význam MCP pro autonomii agentů spočívá v dynamické správě dostupných nástrojů a dat - agent se skrze protokol MCP dokáže automaticky dotázat serveru na seznam dostupných nástrojů a získat k nim přesné definice parametrů. Díky tomu agent v hostitelské aplikaci sám zjistí, jaké nástroje může v rámci své úlohy využívat, a to bez nutnosti explicitního předávání těchto informací. Tento přístup umožňuje agentům být skutečně autonomními - dokážou se adaptovat na

mění se prostředí a využívat nové nástroje, které se objeví, aniž by bylo potřeba měnit jejich základní instrukce či prompt.

2.5 Shrnutí

V této kapitole byl podrobně popsán teoretický základ propojující agilní vývoj, automatizované testování a umělou inteligenci. Byly definovány způsoby specifikace požadavků, které slouží jako primární vstup pro automatizaci, a analyzovány výzvy spojené s údržbou a regresním testováním. Zároveň byly představeny možnosti moderních jazykových modelů a jejich klíčové parametry, jež ovlivňují výsledné chování modelů.

Klíčovým poznatkem kapitoly je vymezení autonomních agentů, kteří díky využití iterativního uvažování a standardizovaného protokolu MCP dokáží překonat limity statického generování a zajistit interaktivní a stabilní testování v dynamickém prostředí webu. Po získání těchto znalostí je možné přejít k návrhu systému, kde budou definovány konkrétní požadavky a architektura řešení, čemuž se podrobně věnuje následující kapitola.

Tato kapitola využívá teoretické poznatky z předchozí kapitoly a přechází k praktickému návrhu systému pro automatizované generování E2E testů s využitím velkých jazykových modelů. Text se v úvodu zaměřuje na analýzu struktury vstupních dat z běžně využívaných ALM nástrojů, z čehož jsou následně odvozeny konkrétní funkční a nefunkční požadavky. Hlavní část kapitoly je poté zaměřena na návrh koncepční architektury - od doménového modelu a distribučního modelu, přes představení a vysvětlení jednotlivých systémových komponent, až po specifikaci exekuční smyčky autonomního agenta. Na závěr kapitoly je pak navržen prototyp uživatelského rozhraní, který zobrazuje navržené řešení a demonstuje jeho klíčové funkce.

3.1 Analýza struktury a sémantiky vstupních artefaktů

Důležitým faktorem pro úspěšnou automatizaci testů je správné pochopení a zpracování vstupních artefaktů, které jsou základem pro generování testů. Jak bylo zmíněno v kapitole 2, tyto artefakty mohou mít různou formu - od textových dokumentů (např. uživatelské příběhy, akceptační kritéria) až po strukturované formáty (např. Gherkin). V ALM nástrojích uživatelské příběhy často nemají jednotnou strukturu, což výrazně ztěžuje jejich zpracování. Zejména nástroje jako Jira nabízejí uživatelům velkou flexibilitu (nejen) při správě uživatelských příběhů. K prozkoumání této problematiky lze využít a nahlédnout na již existující Jira instance, které jsou přístupné veřejnosti. Zde je několik existujících instancí, ze kterých lze získat cenné informace o struktuře a sémantice uživatelských příběhů:

- **Atlassian Jira**¹ - Instance zaměřená na vývoj softwaru samotné společnosti Atlassian (poskytovatel Jira software). Projekty jsou organizovány podle jednotlivých produktů (např. Jira Software, Confluence, Bitbucket...).

¹<https://jira.atlassian.com/secure/BrowseProjects.jspa>

- **Apache Jira**² - Instance zaměřená na vývoj open-source softwaru. Projekty jsou organizovány podle jednotlivých open-source projektů (např. Hadoop, Tomcat, Kafka...).
- **MongoDB Issue Tracker**³ - Instance zaměřená na vývoj databázových systémů. Projekty jsou organizovány logicky dle menších celků (např. MCP server, Shell, drivery dle programovacích jazyků...).
- **Mojang Studios Bug Tracker**⁴ - Instance zaměřená na vývoj her (Minecraft). Jednotlivé projekty jsou dílčí částí her (např. Java Edition, Bedrock Edition, Launcher...).

Před samotnou investigací jednotlivých instancí je vhodné definovat metodiku, jakým způsobem byla data pro analýzu získávána. Vzhledem k obrovskému množství ticketů ve veřejných repozitářích nelze spoléhat na manuální procházení.

Pro průzkum a extrakci relevantních uživatelských příběhů byl využit nativní dotazovací jazyk **JQL (Jira Query Language)**. Tento nástroj umožňuje filtrování napříč projekty na základě mnoha atributů, jako jsou typ úlohy, stav řešení, časové okno vytvoření či fulltextové vyhledávání v obsahu ticketu.

```
project = "TRELLO"  
AND issuetype != Bug  
AND attachments IS NOT EMPTY  
AND assignee IS NOT EMPTY  
AND created >= startOfYear(-1)
```

Zdrojový kód 3.1: Ukázka dotazu v jazyce JQL pro filtrování uživatelských příběhů.

Na ukázce 3.1 je zobrazen dotaz zapsaný v jazyce JQL, který kombinuje několik vlastností:

- **Filtrování dle projektu a typu** - Filtrování ticketů z konkrétního projektu a vyloučení bugů.
- **Validace metadat** - Filtrování ticketů na základě přítomnosti metadat.
- **Časové funkce** - Filtrování ticketů na základě času.

²<https://issues.apache.org/jira/issues/>

³<https://jira.mongodb.org/secure/BrowseProjects.jspa>

⁴<https://bugs.mojang.com/>

Díky těmto dotazům je možné snadno získat relevantní vzorky dat z jakékoliv veřejné instance a následně je porovnat napříč ostatními instancemi.

Zkoumání výše zmíněných veřejných instancí ukázalo, že navzdory využití stejného softwarového nástroje se přístup k evidenci požadavků velmi liší. Tyto rozdíly jsou dány především doménou projektu, firemní kulturou a zavedenými procesy konkrétní organizace.

Metadata ticketů

Typ ticketů není nijak omezen. Napříč instancemi se vyskytují různé druhy ticketů, přičemž každý z nich může mít definovanou svou zcela vlastní strukturu (např. Bug může mít pole pro kroky reprodukce, zatímco User Story pole pro akceptační kritéria). Nejčastěji používanými typy jsou **User Story** a **Bug**. Dále se v rámci instancí vyskytují i další typy jako jsou: New Feature, Task, Improvement, Test, Question, Sub-task, RTC (Request to Change), Epic, Dependency, Suggestion, Support Request, Build Failure, Technical Debt a další.

Co se týče priorit, opět není žádný jednotný systém - a to i v rámci jedné instance. Například v rámci Apache Jira se vyskytují priority jak číselné (P0, P1, P2...), tak i textové (Critical, Major, Minor...). V rámci Atlassian Jira se vyskytují priority pouze textové (Highest, High, Medium, Low). Někdy dochází i k míchání těchto přístupů - např. v rámci MongoDB Jira se vyskytují priority Major - P3, Minor - P4, Trivial - P5, a další.

K určování rozsahů ticketů se využívají různé metriky - od odhadů v rámci story points až po odhady v rámci časových jednotek (hodiny, dny...). Co se týče stavu řešení, i zde není jednotný systém - nejčastější stavy jsou To Do, In Progress, Done, ale v rámci instancí se vyskytuje i mnoho dalších stavů - Ready to Commit, Awaiting Feedback, Blocked. Když je ticket uzavřen, i zde nastává mnoho různých způsobů, jak se uzavření dokumentuje - Closed, Delivered, Resolved, Rejected, Duplicate, Won't Fix, Cannot Reproduce a další.

Description ticketu

V rámci popisu ticketů je společná jedna věc - celý popis je jeden velký textový blok, který může obsahovat jakýkoliv obsah - od čistého textu, přes tabulky a úryvky kódu. Řádná struktura (As a [role], I want [feature], So that [benefit]) se vyskytovala velmi ojediněle - většinou se popis skládal z jednoho odstavce bez jakékoliv struktury či akceptačních kritérií.

Akceptační kritéria

Akceptační kritéria lze v rámci Jira instance definovat jako samostatné pole, ale žádný ze 4 zmiňovaných projektů tuto možnost nevyužívá - to může být způsobeno tím, že se jedná o veřejné repozitáře, kde není rozšířená role Product Ownera, který by měl na starosti definici akceptačních kritérií.

3.2 Specifikace požadavků na systém

Na základě předchozí analýzy vstupních artefaktů a teoretických poznatků jsou v této sekci definovány konkrétní požadavky na navrhovaný systém. Vzhledem k vysoké různorodosti vstupních dat bude systém navržen tak, aby mohl být aplikován na širokou doménu projektů, a to bez nutnosti přizpůsobení pro konkrétní instanci.

3.2.1 Vymezení rozsahu generovaných testů

Systém bude primárně navržen pro generování automatizovaných end-to-end testů s důrazem na ověřování chování uživatelského rozhraní (GUI), obchodní logiku a vizuální interakci s aplikací. Systém naopak nebude koncipován pro verifikaci nefunkčních požadavků, jako jsou výkonnostní testy či bezpečnostní testy. Pokud se v akceptačních kritériích objeví nefunkční požadavky, systém bude instruován, aby se zaměřil výhradně na funkční aspekty - měření časových metrik v rámci E2E testování v prohlížeči může být velmi závislé na výkonu testovacího prostředí (zařízení, síť), což by mohlo vést k nestabilním testům a falešně pozitivním výsledkům.

3.2.2 Funkční požadavky (FR) na systém

Funkční požadavky definují konkrétní chování, které musí systém vykazovat, aby naplnil potřeby uživatelů. Vzhledem k využití velkých jazykových modelů definují nejen samotné vygenerování testů, ale i celý proces - tzv. *agentic workflow*. K určení priorit byla využita metoda MoSCoW (M - Must have, S - Should have, C - Could have, W - Won't have) - výsledné požadavky jsou uvedeny v tabulce 3.1.

ID	Popis požadavku	Priorita
FR-01	Sestavení kontextu: Systém musí přijmout strukturovaná vstupní data (uživatelský příběh a kontext testované aplikace) a transformovat je do vhodné formy agentovi.	Must
FR-02	Explorativní analýza: Před generováním kódu agent proaktivně naviguje testovanou aplikací a zkoumá strukturu DOMu pomocí dostupných nástrojů.	Must
FR-03	Křížová verifikace: Agent musí systematicky porovnat každé akceptační kritérium se skutečným chováním aplikace.	Must
FR-04	Řízené selhání: Při nefunkční či neimplementované funkcionalitě agent vygeneruje test s očekávaným správným chováním, ale označí jej speciálním příznakem.	Should
FR-05	Standardizace výstupu: Vygenerovaný kód musí být validní, přeložitelný a spustitelný.	Must
FR-06	Popis implementace: Vygenerovaný test by měl obsahovat krátké shrnutí, které popisuje výsledky verifikace oproti akceptačním kritériím.	Should
FR-07	Vizualizace procesu: Uživatelské rozhraní by mělo zobrazovat flow agenta a vhodně jej vizualizovat uživateli.	Should
FR-08	Exekuce testů: Systém by mohl umožňovat přímé spuštění vygenerovaných testovacích scénářů z uživatelského rozhraní bez nutnosti přepínání do jiného prostředí (např. IDE, CLI).	Could
FR-09	Regenerace na základě zpětné vazby: Systém by mohl podporovat regeneraci testů - v případě selhání běhu testu by měl agent přijmout chybový report a na jeho základě opravit testovací kód.	Could
FR-10	Dynamická konfigurace agenta: Uživatel by mohl mít možnost upravovat konfiguraci agenta dle potřeby - volba LLM modelu či konkrétních toolů.	Could

Tabulka 3.1: Specifikace funkčních požadavků na systém.

3.2.3 Nefunkční požadavky (NFR) na systém

Nefunkční požadavky se zaměřují na kvalitu, výkon, bezpečnost a spolehlivost celého systému. V tabulce 3.2 jsou uvedeny klíčové nefunkční požadavky, které musí

být zohledněny při návrhu a dodrženy při implementaci.

ID	Popis požadavku	Priorita
NFR-01	Architektura MCP: Logika agenta musí být oddělena od fyzického ovládání prohlížeče pomocí standardizovaného MCP protokolu.	Must
NFR-02	Odolnost vůči selhání: Pokud volání nástroje selže, agent se nesmí zacyklit, ale musí chybu reflektovat či zvolit jiný přístup k vyřešení úlohy.	Must
NFR-03	Bezpečnost API klíčů: Citlivé autentizační údaje musí být bezpečně spravovány backendovou částí aplikace.	Must
NFR-04	Modularita modelů: Systém by měl umožňovat snadné přidání nových LLM modelů bez významného vlivu na celkovou architekturu.	Should
NFR-05	Rozšiřitelnost nástrojů: Architektura by mohla umožňovat snadné přidávání nástrojů bez nutnosti refaktORIZACE jádra agenta.	Could

Tabulka 3.2: Specifikace nefunkčních požadavků na systém.

3.3 Koncepční architektura systému

3.3.1 Doménový model aplikace

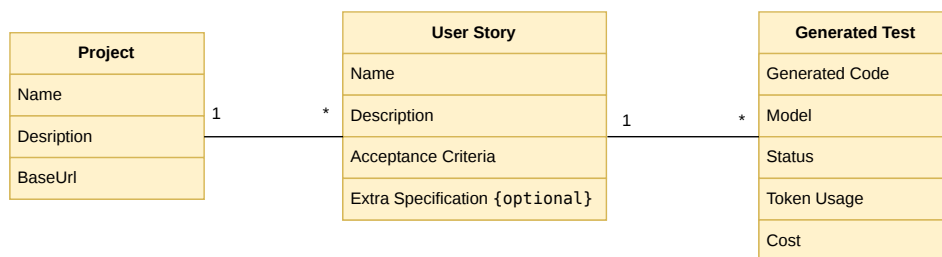
Jak ukázala analýza veřejných instancí ALM nástrojů (viz sekce 3.1), vstupní artefakty napříč projekty a odvětvími postrádají jednotnou strukturu. Pokud by agent přistupoval přímo k surovým datům z těchto různorodých systémů, byla by extrakce informací velmi nespolehlivá a náročná. Tento systém proto zavádí vlastní, normalizovaný doménový model, který funguje jako standardizovaná mezivrstva mezi nestrukturovanými daty z externích nástrojů a čistým kontextem, který bude agentovi předán. Díky tomu bude možné zajistit, že agent bude pracovat s předvídatelným a konzistentním formátem dat, což výrazně zlepší spolehlivost a efektivitu generování testů (čímž naplňuje požadavek FR-01).

Struktura obsahuje tři klíčové entity:

1. **Projekt** - Reprezentuje konkrétní testovanou aplikaci. Obsahuje její základní informace (název, popis, URL).
2. **Uživatelský příběh** - Reprezentuje jeden konkrétní ticket. Obsahuje relevantní informace pro generaci testů - název, popis, akceptační kritéria a nepovinný dodatečný kontext (např. odkazy na dokumentaci, konkrétní požadavky na testy...).

3. **Vygenerovaný test** - Reprezentuje testovací scénář vygenerovaný agentem. Kromě samotného kódu obsahuje i metadata o generování testu (název, použitý model, množství tokenů, celková cena).

Model je znázorněn na obrázku 3.1. Mezi entitami jsou definovány relace - mezi projektem a uživatelským příběhem je relace 1:N - projekt může mít více uživatelských příběhů, ale každý uživatelský příběh patří pouze jednomu jedinému projektu. V praxi by se našly i výjimky, kdy by uživatelský příběh mohl být sdílen mezi více projekty (např. společná funkcionality či separátní projekt pro frontend a backend), ale dle dříve zanalyzovaných ticketů (které v drtivé většině případů obsahovaly vazbu pouze na jeden projekt) a pro zjednodušení modelu byla tato možnost vyloučena. Relace mezi uživatelským příběhem a vygenerovaným testem je také 1:N - jeden uživatelský příběh může mít více vygenerovaných testů, ale každý vygenerovaný test patří pouze jednomu jedinému uživatelskému příběhu.



Obrázek 3.1: Doménový model systému.

Ostatní prvky identifikované v analýze (např. metadata ticketů) jsou záměrně vynechány, protože pro generování testů nepřinášejí žádnou přidanou hodnotu a navíc jejich zahrnování v agentově kontextu by mohlo vést k zvětšení množství tokenů a tím i nákladů na generování testů.

3.3.2 Distribuce a perzistence dat

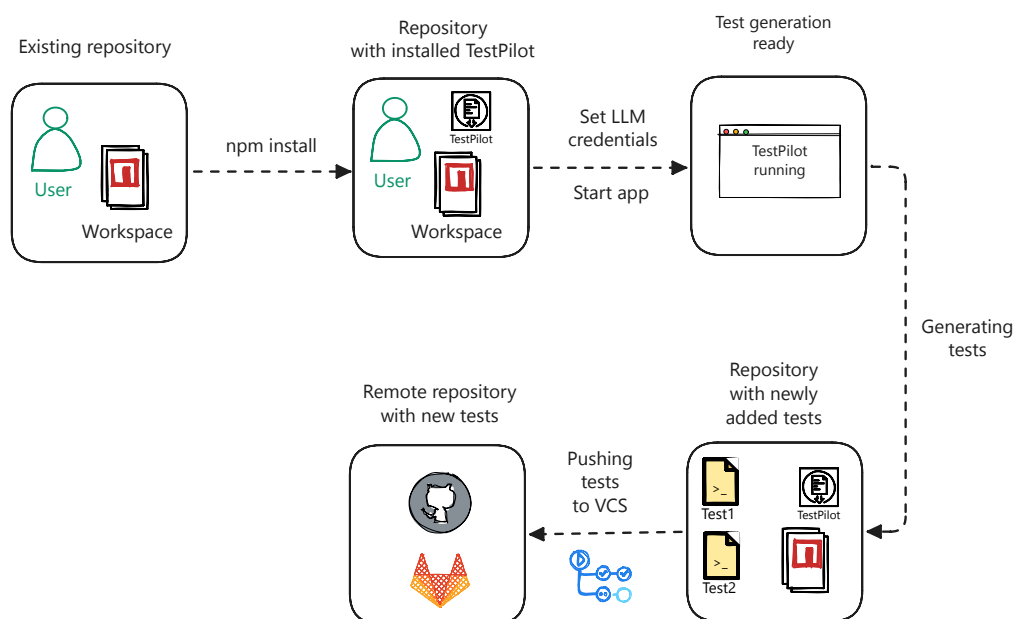
Před samotnou definicí architektury agenta je nejprve nutné rozhodnout, jaký způsob distribuce systému zvolit a jakým způsobem ukládat data. Původní myšlenkou bylo využít tradiční architekturu s nasazením relační databáze (např. PostgreSQL). Výhodou by bylo, že veškeré testy a uživatelské příběhy by byly uloženy centralizovaně v jedné konkrétní databázi, což by umožnilo snadné sdílení a přístup napříč uživateli a vývojovými týmy. Z tohoto přístupu se nakonec ustoupilo kvůli následujícím důvodům:

- **Ukládání výsledných testů** - Největší úskalí by bylo definování tzv. *source of truth* pro vygenerované testy. Pokud by se testy ukládaly do centrální databáze, vznikla by potřeba synchronizace mezi touto databází a prostředními, kde by se testy používaly (např. IDE, CI/CD pipeline). Databáze by tak obsahovala veškeré vygenerované testy a bylo by možné trasovat, kdy a kým byly testy vygenerovány, ale zároveň by stejně bylo potřeba testy ukládat do repozitářů, aby bylo možné je spouštět a nakonec by stejně zdrojem pravdy pro testy zůstaly repozitáře.
- **Správa databáze** - S produkčním nasazením by bylo potřeba zajistit správu a údržbu databáze, což by vývojáře odvádělo od hlavního cíle - jednoduché generování testů.
- **Závislost na třetí straně** - Program by byl závislý na další externí službě - k závislosti na dostupnosti LLM modelů by se tak přidala i závislost na dostupnosti a správě databáze. Navíc pokud by aplikace běžela jako samostatná služba, vývojáři by museli z vývojových prostředí otevírat další rozhraní, což by mohlo vést k nižší adopci nástroje.

Tato analýza vede k závěru, že tradiční přístup s databází by nedisponoval uživatelskou přívětivostí, která je pro adopci nástroje klíčová. Všechny výše zmíněné nevýhody řeší tzv. *local-first* přístup, který umožňuje vývojářům pracovat s nástrojem přímo z jejich lokálního prostředí bez nutnosti přepínání do jiných rozhraní či závislosti na externích službách. Systém bude distribuován jako samostatný instalovatelný balíček (prostřednictvím správce balíčků NPM - Node Package Manager), který si uživatel pouze nainstaluje do svého projektu, kde chce testy generovat (případně může jednou globálně, pokud chce testy generovat napříč vícero projekty). Následně jen pomocí jednoduchého CLI příkazu spustí systém lokálně a celý proces generování testů proběhne přímo na jeho zařízení. Jediné nastavení, které bude muset provést, je konfigurace přístupových údajů k LLM modelům, což bude záležitost několika řádků v konfiguračním souboru, který bude umístěn lokálně v projektu.

Díky tomu budou mít vývojáři systém přímo ve svém IDE bez nutnosti otevírání dalších rozhraní, *source of truth* bude přímo v repozitáři, do něhož bude aplikace testy ukládat a bude je moci i číst a rovnou spouštět, a navíc veškerá historie generovaných testů, uložených projektů a uživatelských příběhů, bude uložena v souboru, který bude možno distribuovat přes Git.

Celé vývojářské flow, které bylo popisováno, je zobrazeno na obrázku 3.2, kde navržený systém je označován jako TestPilot.



Obrázek 3.2: Vývojářské flow s využitím local-first přístupu.

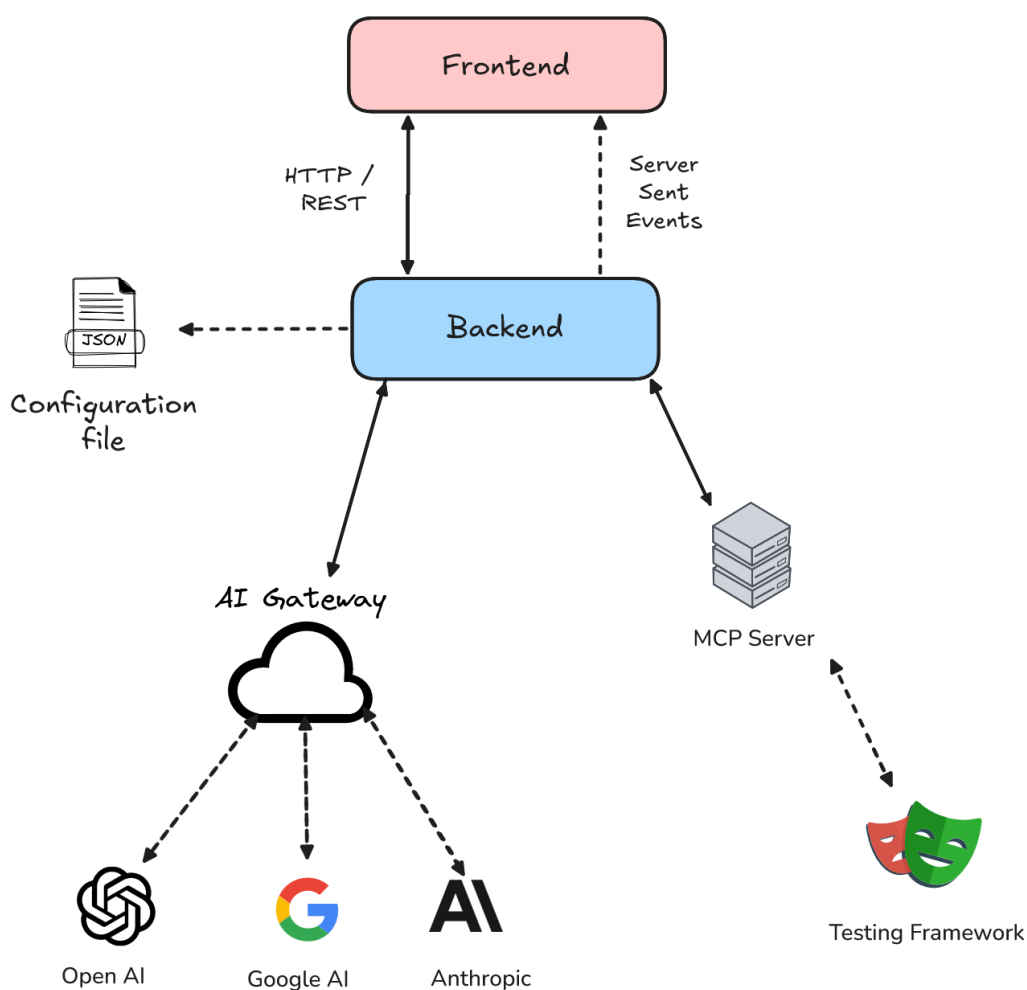
3.3.3 Architektura systémových komponent

Nejprve v úvahu přicházela možnost navrhnout systém jako čistě backendovou službu, která by běžela jen v terminálu a komunikovala s uživatelem pouze prostřednictvím CLI, ale tento přístup by nebyl uživatelsky přívětivý pro ne-technické uživatele. Na základě toho bude systém navržen jako *full-stack* aplikace, která bude mít intuitivní GUI a umožní tak širší adopci napříč různými uživateli a týmy.

Schéma architektury bude rozděleno do 4 hlavních logických částí a jejich vzájemná komunikace je znázorněna na obrázku 3.3.

Frontend (Uživatelské rozhraní)

Tato klientská část zajišťuje interakci s uživatelem a slouží jako hlavní bod pro konfiguraci a vizualizaci celého procesu generování testů. S backendem bude komunikovat pomocí dvou odlišných komunikačních vzorů. Pro standardní operace čtení a zápisu dat (např. zakládání projektů, správa uživatelských příběhů) bude využíváno klasické HTTP/REST rozhraní. Pro sledování myšlenek agenta při generování ale synchronní HTTP komunikace není ideální, protože potřebujeme průběžně aktualizovat uživatelské rozhraní o nové informace (co agent dělá, jaké nástroje volá). Proto bude využit jednosměrný protokol Server-Sent Events (SSE), který umožňuje



Obrázek 3.3: Koncepční architektura systému.

efektivní streamování dat z backendu na frontend v reálném čase a je alternativou k WebSocketům, které jsou pro tento účel komplexnější.

Backend

Nejdůležitější část celé architektury - její hlavní zodpovědností je řízení celého procesu generování testů, udržování agentní smyčky a zprostředkování komunikace mezi jazykovými modely, testovacím frameworkem a uživatelským rozhraním. Bude zodpovědný také za bezpečné čtení a zápis dat do lokálního konfiguračního souboru.

AI Gateway

Zajištění spolehlivé komunikace s velkými jazykovými modely představuje zásadní výzvu. Každý poskytovatel (provider) umělé inteligence (OpenAI, Anthropic, Google) využívá pro své API zcela odlišná vstupní a výstupní schémata - liší se formáty pro definici dostupných nástrojů (tool calling), což je pro správnou funkčnost agenta klíčové. Dále se liší i strukturováním historie zpráv či výpočtem spotřebovaných tokenů - další klíčový faktor pro správné fungování agenta. Přímá integrace těchto jednotlivých API by vedla k vysoké komplexitě zdrojového kódu a obtížné udržitelnosti. Pro každého nového poskytovatele by bylo potřeba naimplementovat vlastní logiku pro překlad požadavků a zpracování odpovědí. Systém proto zavede dedikovanou vrstvu AI Gateway, která tyto rozdíly efektivně abstrahuje za pomoci existujících řešení (např. knihovny AI Vercel SDK) a díky níž bude budoucí rozšiřitelnost o nové modely výrazně jednodušší - bude pouze třeba přidat existující adaptér pro daného poskytovatele, aniž by bylo potřeba zasahovat do zbytku architektury či psát ručně překlad pro každý model.

MCP Server a testovací framework

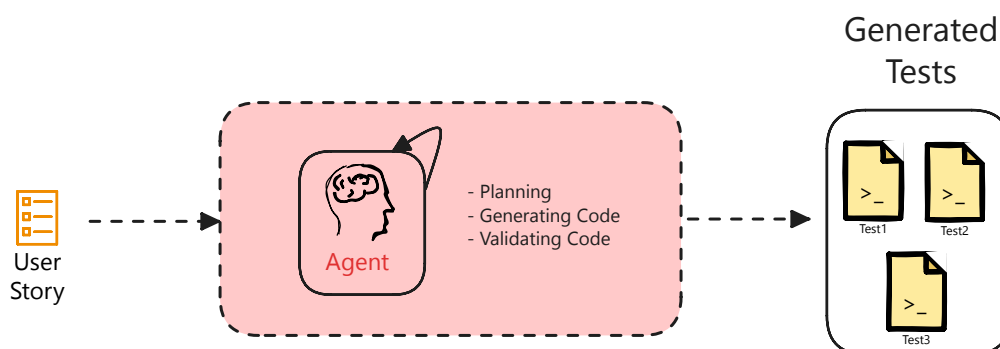
Pro zajištění modularity a bezpečnosti systému by autonomní agent neměl mít přímou kontrolu nad prohlížečem z backendu. Místo toho budou požadavky na interakci s prohlížečem odesílány do dedikovaného serveru, který bude využívat standardizovaný Model Context Protocol a poběží jako zcela samostatný izolovaný proces. Tento server bude přijímat příkazy od agenta (např. kliknutí na tlačítko, vyplnění formuláře) a překládat je do konkrétních instrukcí pro zvolený testovací framework. Framework poté ovládá webový prohlížeč a vrací zpět výsledky akce, které backend předá agentovi pro další zpracování.

3.3.4 Architektura agenta a exekuční smyčka

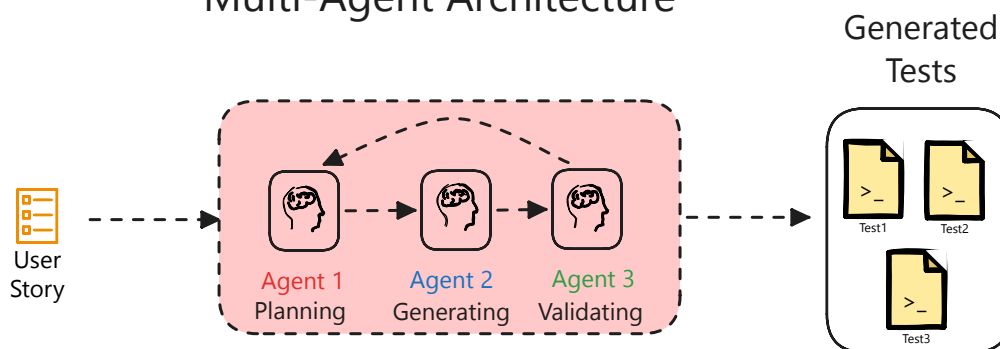
Zdůvodnění Single-agent architektury

Jak popisuje příručka OpenAI [41], existují dva hlavní přístupy k návrhu agentních systémů - **Single-agent** a **Multi-agent** architektura, které jsou zobrazeny na obrázku 3.4.

Single-Agent Architecture



Multi-Agent Architecture



Obrázek 3.4: Schéma Single-agent vs Multi-agent architektury.

Single-agent architektura se zaměřuje na jednoho centrálního agenta, který má přístup ke všem nástrojům a informacím potřebným k vykonání úkolu. Tento agent je zodpovědný za celý proces - od plánování, přes exekuci, až po generování výstupu. Tento přístup je jednodušší na implementaci, snadněji se ladí a chová se determinističtěji, protože veškeré rozhodování je centralizované v jednom agentovi.

Oproti tomu multi-agentní architektura rozděluje úkol mezi více specializovaných agentů, kde každý z nich má svou specifickou roli - jeden může být zodpovědný za plánování, druhý za generování kódu a třetí za revizi kódu. Tento přístup může být efektivní pro řešení komplexnějších úloh, ale přináší s sebou vyšší míru nedeterminismu (jelikož agenti mohou mít různé strategie), je náchylný k zacyklení (nekonečné debaty mezi jednotlivými agenty) a jeho debugging je složitý, jelikož je třeba sledovat flow mezi více entitami. Navíc tento způsob může generovat obrovskou režii v podobě spotřebovaných tokenů právě díky komunikaci mezi agenty, která vyžaduje další volání modelů.

Proto systém bude využívat striktní Single-agent architekturu, kde bude definován agent s rolí *Senior QA Automation Engineer*, který bude provádět celý proces postupně - od prvotní navigace, přes ověření stávajícího stavu aplikace až po finální generování kódu.

ReAct smyčka

Jak bylo popsáno v kapitole dedikované analýze LLM agentů, systém bude využívat paradigma ReAct. V rámci fáze Thought bude agent analyzovat aktuální stav aplikace a plánovat další krok, který by měl podniknout. V Action fázi bude volat konkrétní nástroje prostřednictvím MCP serveru, aby získal potřebné informace o stavu aplikace. V Observation fázi bude backend zpracovávat výsledky těchto akcí a poskytovat jejich zpětnou vazbu agentovi, který na základě toho bude pokračovat v iteraci, dokud nebude mít dostatek informací na vygenerování finálního testu.

Ochranné mechanismy

K snížení rizika halucinací bude agent instruován sadou striktních pravidel, která bude muset dodržovat. Bude se jednat o následující mechanismy:

- **Tool Enforcement** - Agent bude muset použít nástroje pro průzkum aplikace předtím, než bude moci vygenerovat finální test. Běžně se stává, že agenti se snaží rovnou vygenerovat test “naslepo”, aniž by měli dostatek informací o stavu aplikace.
- **Code Validation** - Jakmile agent vygeneruje kód, bude muset projít základní validací, aby se ověřilo, že je syntakticky správný a přeložitelný. Pokud ne, bude agent vyzván k opravě kódu, dokud nebude kód validní.
- **Circuit Breaker** - Aby se předešlo nekonečnému zacyklení (které by vyústilo v nekonečné volání LLM modelů), bude implementován mechanismus, který po určitém počtu po sobě jdoucích selhání nástrojů přeruší agentní smyčku a ukončí proces generování testu.

Tento přístup by měl zlepšovat spolehlivost a kvalitu vygenerovaných testů a zároveň minimalizovat náklady spojené s opakovanými voláními modelů.

3.4 Návrh uživatelského rozhraní

Celkové rozvržení a UX principy

Aplikace bude koncipována po vzoru moderních vývojových prostředí (Visual Studio Code, JetBrains IDEs), které jsou pro vývojáře dobře známé a intuitivní. Na

obrázku 3.5 je zobrazen návrh uživatelského rozhraní, které bude rozděleno do tří hlavních částí - levý navigační panel, hlavní pracovní plocha a pravá vizualizační část.

The interface is divided into three main sections:

- Project section (Left):** Contains a 'Prj 1' dropdown menu, two empty square buttons, a list of stories (Story 1, Story 2, Story 3, Story 4), and a 'Settings' icon at the bottom.
- Main workspace (Center):** Contains a form with fields for 'Name', 'Description', 'Acceptance Criteria', and 'Extra Specification'. Below the form are two buttons: 'Generate' (green) and 'Terminate' (red). At the bottom, it displays 'Model: Claude Opus-4.6' and 'Active Tools: tool1, tool2'.
- Test section (Right):** Contains an 'Activity Log' tab, 'Total Cost: 1\$' and 'Token Usage: 180k' statistics, a list of steps (Step 1: Thinking, Step 2: Tool Call, Step 3: Tool Result, Step 4: Code Validation, Step 5: Completed), and a 'Generated Code' block showing a Python test snippet:


```
import {test} from 'test'
test() {
  assert(a,b)
}
```

Obrázek 3.5: Návrh uživatelského rozhraní.

Využit bude boční navigační panel na levé straně, který bude obsahovat sekce pro správu projektů a k nim přiřazených uživatelských příběhů. Zároveň by mohl také obsahovat proklik do nastavení, kde by se mohly upravovat konfigurace agenta (volba LLM modelu, volba nástrojů).

Hlavní pracovní plocha bude sloužit pro specifikaci uživatelských příběhů - bude zde zobrazen formulář, díky němuž půjdou snadno vytvářet a případně editovat uživatelské příběhy. Pod tímto formulářem by se měla nacházet i sekce pro spuštění generování testů a případně i ovládací prvek pro zastavení běhu agenta. Viditelný by měl být i aktuálně využívaný model (aby uživatel nemusel otevírat nastavení) a případně využívané nástroje.

Pravá část plochy bude sloužit pro vizualizaci všeho ohledně testů - od samotného průběhu generování (co agent dělá), po výsledný kód vygenerovaných testů a případně živý náhled do lokálního souborového systému, kde jsou testy uloženy.

Vizualizace procesu agenta

Tato sekce bude klíčová pro zajištění transparentnosti a důvěry uživatelů v systém. Bude zobrazovat celý průběh generování testů - od prvotního zamyšlení agenta, přes volání nástrojů až po finální validaci a generaci kódu. Pokud to daný krok umožní, bude zobrazovat i konkrétní výstup (např. výsledek tool callu). Po vygenerování testu bude (kromě samotného kódu) zobrazeno i krátké shrnutí v podobě počtu zkonsumovaných tokenů, celkové ceny a případně i možnost regenerovat test na základě zpětné vazby od uživatele.

Sekce pro spuštění testů a zobrazení výsledků

Tato sekce by měla nahradit potřebu přepínat mezi běžícím systémem pro generování testů a vývojovým prostředím, čímž se vývojářům ušetří čas. Uživatel by měl mít možnost vidět a spravovat testy, které má v aktuálním repozitáři - a pokud možno, tak je rovnou i spouštět a vidět výsledky jejich běhu.

3.5 Shrnutí

V této kapitole byl představen kompletní návrh softwarového řešení včetně uživatelského rozhraní. Na základě analýzy artefaktů byl vytvořen vlastní normalizovaný doménový model a definována striktní pravidla pro minimalizaci rizika halucinací jazykových modelů v agentské smyčce. Pro maximální uživatelskou přívětivost a adopci byl zvolen *local-first* distribuční model, který spolu s MCP serverem a AI Gateway představuje základ pro udržitelnou a snadno rozšiřitelnou architekturu. Definovány byly i konkrétní funkční a nefunkční požadavky, které budou později sloužit k ověření úspěšnosti implementace a naplnění cílů této práce. Konkrétní technologie, které budou pro implementaci tohoto systému použity, jsou předmětem další kapitoly, která se zaměří na výběr vhodného technologického stacku.

Použité technologie

4

Pro transformaci navržené architektury do podoby reálného softwaru bylo nezbytné vhodně zvolit technologie, které bude systém využívat. Tato kapitola představuje hlavní nástroje, knihovny a frameworky, na kterých bude systém postaven. Cílem výběru nebylo pouze využití moderních technologií, ale primárně nalezení kombinace, která zajistí vysokou míru typové bezpečnosti, vzájemnou kompatibilitu a nativní podporu pro specifické požadavky autonomního agenta.

4.1 TypeScript

Jako primární programovací jazyk celého systému byl zvolen TypeScript. Jedná se o striktně typovanou nadstavbu jazyka JavaScript, která do webového vývoje přináší statickou kontrolu typů [44, 45]. Hlavní výhodou této technologie je schopnost detekovat typové a strukturální chyby již během fáze vývoje a kompilace, na rozdíl od JavaScriptu, který tyto chyby odhalí až při samotném běhu aplikace. Tato schopnost je pro navrhovaný systém klíčová, protože vzhledem k neustálé a dynamické výměně dat mezi frontendem, backendem a velkými jazykovými modely je zajištění konzistentních datových struktur naprosto nezbytné pro správnou funkčnost, spolehlivost a udržitelnost.

To bude realizováno prostřednictvím DTO (Data Transfer Object) patternu, kdy budou definovány konkrétní datové typy a rozhraní pro všechny klíčové entity systému (projekty, uživatelské příběhy, vygenerované testy). Díky tomu bude možné sdílená rozhraní definovat pouze jednou napříč celou aplikací, což výrazně zjednodušuje vývoj, eliminuje redundantní kód a snižuje riziko chyb.

4.2 React

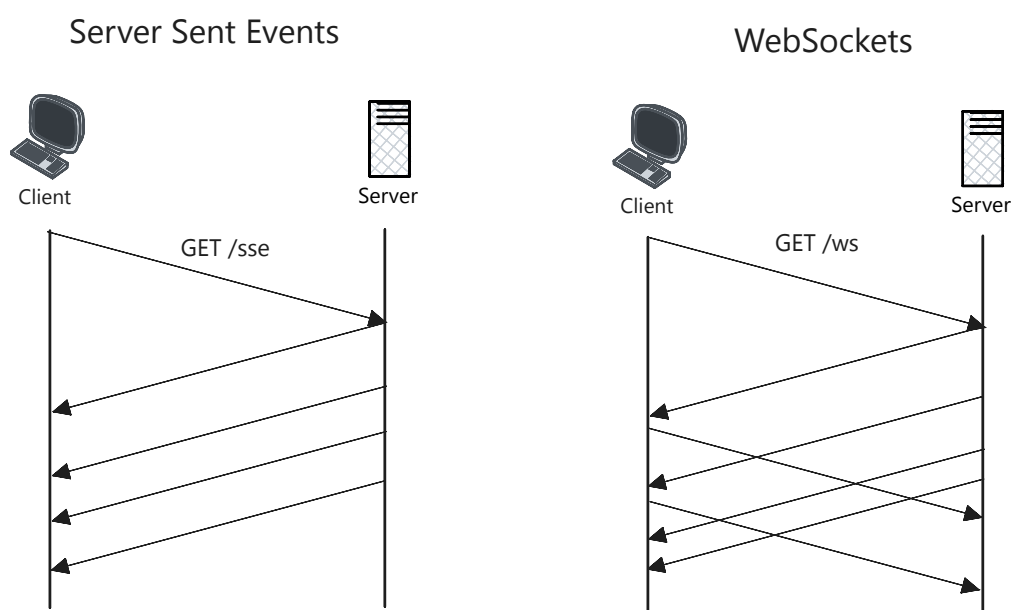
Pro vývoj uživatelského rozhraní byla zvolena populární javascriptová knihovna React. Disponuje deklarativním a komponentovým přístupem, díky němuž je možné rozdělit komplexní rozhraní do menších, znovupoužitelných částí. Vzhledem k důrazu na maximální interaktivitu navrhovaného systému (zejména pro dynamické vy-

kreslování myšlenkových pochodů agenta, správu stavu rozhraní pro specifikaci uživatelských příběhů či zobrazení výsledků vygenerovaných testů) je efektivní správa stavu komponent klíčová. Zásadním benefitem Reactu je využití tzv. virtuálního DOMu (Virtual DOM), který zaručuje, že se při změně stavu překreslí pouze ta část uživatelského rozhraní, jež se skutečně změnila [46]. To výrazně zlepšuje výkon a zajišťuje plynulý chod aplikace i při velkém počtu příchozích událostí. Dalším rozhodujícím faktorem pro volbu této knihovny byla podpora jazyka TypeScript prostřednictvím syntaxe TSX, což umožňuje bezproblémově napojit frontend na sdílené DTO modely a zachovat striktní typovou bezpečnost napříč celou aplikací.

4.3 NestJS

Pro vývoj backendové části aplikace byl zvolen framework NestJS založený na platformě Node.js. Tento framework byl vybrán především pro svou modulární architekturu a integrovanou podporu pro *Dependency Injection* - klíčový návrhový vzor pro udržitelnost a testovatelnost kódu [47]. Vzhledem k tomu, že agent bude obsahovat komplexní logiku, která zahrnuje komunikaci s LLM modely, správu lokálního souborového systému, správu uživatelských příběhů a projektů, je důležité, aby tyto domény byly striktně logicky oddělené do samostatných modulů, což povede k lepší čitelnosti, testovatelnosti a dlouhodobé udržitelnosti celého backendu. NestJS navíc nabízí nástroj `Nest CLI`, který umožňuje rychlé generování boilerplate kódu pro nové moduly, služby a kontrolery, což výrazně usnadňuje a zrychluje vývoj. Na rozdíl od klasického Express.js podporuje nativně i asynchronní streamování dat pomocí SSE, díky čemuž je eliminována potřeba implementovat komplexnější řešení na bázi WebSocketů.

Jak je vidět na obrázku 4.1, pro tento případ použití je SSE vhodnou volbou. Zatímco WebSokety vyžadují udržování obousměrného spojení, SSE funguje na principu standardního HTTP požadavku, kdy klient pouze otevře spojení (`GET /sse`) a následně server už jen jednosměrně odesílá průběžné aktualizace směrem k uživateli.



Obrázek 4.1: Schéma komunikace pomocí SSE vs WebSocketů.

4.4 Vercel AI SDK

Pro komunikaci s velkými jazykovými modely (LLM) byla zvolena knihovna Vercel AI SDK. Jak již bylo zmíněno v předchozí kapitole, největším úskalím integrace různých poskytovatelů umělé inteligence je značná rozdílnost specifikací jejich API. Vercel AI SDK tento problém elegantně řeší tím, že funguje jako univerzální adaptér. Poskytuje jednotné rozhraní, díky kterému lze, jak uvádí Patil et al. ve studii *Real time inventory management system powered by generative user interface* [48], plynule přecházet mezi různými modely pouhou úpravou konfigurace, aniž by bylo nutné zasahovat do vnitřní logiky agenta.

Zásadním důvodem ale byla podpora pro volání nástrojů - knihovna nativně řeší překlad definic nástrojů (pomocí JSON schémat) do formátů specifických pro jednotlivé poskytovatele a automaticky parsuje a validuje odpovědi od modelů zpět do striktně typovaných TypeScript objektů.

V neposlední řadě nabízí i spoustu předpřipravených hooků pro React/Next.js aplikace a podporuje streamování odpovědí.

4.5 Playwright a Model Context Protocol (MCP)

Jako finální testovací framework byl zvolen Playwright. Jak popisuje studie *Automated end to end testing with Playwright for React applications* [49], Playwright efektivně řeší specifické výzvy moderních dynamických aplikací, jako jsou asynchronní operace, složité překreslování komponent a komplexní správa stavu. Přestože byly základní výhody tohoto nástroje představeny v teoretické části, níže jsou na základě zmíněné studie shrnuty ty nejzásadnější, které rozhodly o jeho výběru pro tento systém:

- **Pokročilý Auto-waiting a stabilita:** Framework automaticky vyčkává na interaktivitu prvků, čímž eliminuje potřebu statických čekacích dob a radikálně snižuje nestabilitu testů.
- **Network Interception a API Mocking:** Nativní podpora zachytávání síťového provozu umožňuje simulovat chybové stavy a podvrhovat API odpovědi bez nutnosti úprav na skutečném backendu.
- **Složité uživatelské interakce:** Nástroj spolehlivě zvládá práci se Shadow DOM, testování napříč více záložkami a komplexní akce typu *drag-and-drop*.

Pro integraci s autonomním agentem je navíc klíčová dostupnost NPM balíčku `@playwright/mcp`, který běží jako samostatný proces a komunikuje s backendem přímo přes protokol MCP.

4.6 Shrnutí

V této kapitole byl představen a zdůvodněn výběr technologií, které budou použity pro implementaci navrženého systému. Zvolený technologický stack sahá od striktní typové kontroly (TypeScript), přes reaktivní uživatelské rozhraní (React) a modulární backend (NestJS), až po specializované nástroje pro komunikaci s LLM modely (Vercel AI SDK) a testování webových aplikací (Playwright s podporou MCP protokolu). Konkrétní použití těchto technologií bude detailně představeno v další kapitole, jež se bude věnovat samotné implementaci.

Implementace

5

TODO

5.1 Struktura a organizace projektu

Celý systém byl implementován jako tzv. monorepo (monolithic repository) s využitím nástroje Turborepo, který umožňuje spravovat více logicky oddělených aplikací, knihoven či služeb v rámci jednoho repozitáře. Díky tomu je možné sdílet kód, typy a rozhraní napříč různými částmi systému bez nutnosti publikovat a spravovat samostatné balíčky pro každou z těchto částí. Nástroj Turborepo navíc optimalizuje proces sestavování (build process) díky inteligentnímu cachování, které zajišťuje, že se znovu sestaví pouze ty části systému, které byly skutečně změněny, což výrazně zrychluje vývoj a testování.

Základní struktura projektu je rozdělena do dvou hlavních adresářů - složka `apps` pro spustitelné aplikace (backend a frontend) a složka `packages` pro sdílené knihovny (např. pro DTO modely, CLI nástroje). Zjednodušené zobrazení této struktury je znázorněno níže:

```
└─ testpilotai
   └─ apps
      ├── api ..... Backendová část aplikace (NestJS)
      └── client ..... Klientské rozhraní (React)
   └─ packages
      ├── cli ..... Knihovna pro distribuci přes CLI
      └── shared ..... Sdílené typy, rozhraní a DTO modely
   └─ package.json .... Konfigurace projektu a závislostí
   └─ turbo.json ..... Konfigurace Turborepo pipeline
```

Důležitou částí této struktury je složka `./packages/shared`, která obsahuje veškerá doménová data, komunikační DTO modely a definice typů. Jelikož je tato logika využívána jak backendem (`api`), tak frontendem (`client`), je zaručeno, že obě

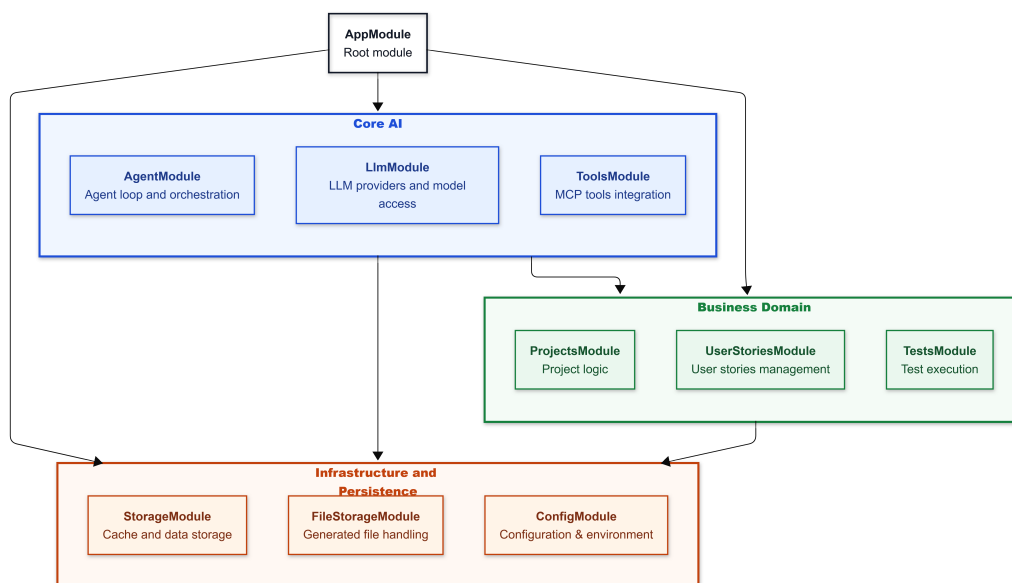
vrstvy komunikují pomocí identických datových struktur a že jakákoliv změna v datovém modelu se díky TypeScriptu okamžitě projeví chybou při kompilaci, čímž se výrazně snižuje riziko neúmyslných chyb a naplňuje tak požadavek na striktní typovou bezpečnost napříč celou aplikací.

5.2 Implementace backendové části

Zatímco předchozí sekce nastínila celkovou strukturu projektu a jeho uspořádání, tato část se zaměří na konkrétní implementaci backendové části z vyšší perspektivy. Cílem je představit logickou architekturu serveru, klíčové moduly a jejich zodpovědnosti - konkrétní detaily implementace budou představeny v rámci dalších sekcí.

5.2.1 Doménové moduly

Backendová část (apps/api) je postavena na frameworku NestJS a dodržuje jeho modulární architekturu. Aplikace je rozdělena do nezávislých doménových modulů, které jsou pak jako jednotlivé funkční celky sdružené v hlavním vstupním bodě (AppModule). Architektura je logicky vrstvena do tří hlavních domén, které jsou zobrazeny na obrázku 5.1.



Obrázek 5.1: Zjednodušený diagram vrstvených závislostí backendových modulů.

Díky této separaci je zajištěno, že každý modul má jasně definovanou zodpovědnost, což výrazně zlepšuje nejen čitelnost a udržitelnost kódu, ale i jeho testova-

telnost. Každý modul může být jednoduše testován izolovaně pomocí unit testů a následně i integračních testů, které ověří správnou spolupráci mezi moduly.

Tok dat (Request/Response flow) uvnitř backendu je striktně vrstvený a dodržuje best practices zapouzdření. Požadavek z klientské aplikace je nejdříve zachycen v příslušném kontroleru dané domény (např. `AgentController`), který provádí počáteční validaci vstupních parametrů pomocí DTO a Zod schémat. Kontroler nikdy nekomunikuje s konkrétními službami z jiných modulů - od toho je tu dedikovaná servisní vrstva (např. `AgentOrchestrationService`), která zapouzdřuje veškerou business logiku. Až tato vrstva deleguje dílčí úkoly dál a bezpečně komunikuje se specializovanými službami (Services) z jiných modulů.

Core AI Orchestration

Tato vrstva tvoří jádro celého systému a obsahuje moduly zodpovědné za interakci s velkými jazykovými modely a jejich orchestraci. `AgentModule` implementuje řídicí smyčku agenta a orchestrace běží přes služby `AgentService` a `AgentOrchestrationService`. Zároveň tento modul přímo integruje `LlmModule`, `ToolsModule`, `ProjectsModule`, `UserStoriesModule`, `StorageModule` a `FileStorageModule`, což z něj dělá hlavní integrační bod backendu. `LlmModule` poskytuje jednotné rozhraní pro volání různých poskytovatelů modelů přes AI Gateway, `ToolsModule` pak dále zajišťuje načtení dostupných nástrojů a jejich exekuci přes MCP vrstvu (aktuálně s podporou Playwright nástrojů).

Business Domain Modules

Tyto moduly reprezentují doménovou logiku aplikace. `ProjectsModule` zajišťuje správu projektů a jejich konfigurací, `UserStoriesModule` spravuje uživatelské příběhy a jejich artefakty (název, popis, akceptační kritéria), a `TestsModule` zajišťuje načítání, spouštění a mazání již vygenerovaných Playwright testů. Samotné generování testů je řízeno přes endpointy modulu `AgentModule`; `TestsModule` slouží primárně pro exekuci a správu výsledných testovacích souborů.

Infrastructure & Persistence

Nejnižší vrstva zajišťuje veškeré nefunkční aspekty - od konfigurace přes management dat v cache až po bezpečný zápis do souborového systému. `ConfigModule` zajišťuje načtení proměnných prostředí a práci s lokálním konfiguračním souborem, `StorageModule` ukládá metadata o generování do souboru. `FileStorageModule` se stará o zápis vygenerovaných testů do cílového adresáře, vytváření složek a řešení kolizí názvů souborů.

```
1 export interface Project {  
2   id: string;  
3   name: string;  
4   description: string;  
5   baseUrl: string;  
6   createdAt: string;  
7   updatedAt: string;  
8 }  
9  
10 export interface UserStory {  
11   id: string;  
12   projectId: string;  
13   title: string;  
14   description: string;  
15   acceptanceCriteria: string;  
16   extraInfo: string;  
17   createdAt: string;  
18   updatedAt: string;  
19 }
```

Zdrojový kód 5.1: Ukázka sdílených doménových entit v balíčku @testpilotai/shared.

5.2.2 Implementace doménového modelu a sdílených DTO

Navrhovaný doménový model z kapitoly 3 byl v rámci implementace převeden do sdílených TypeScript interfaců, které jsou umístěny v balíčku packages/shared a přímo využívány jak frontendem, tak backendem. Struktura balíčku shared je rozdělena do dvou hlavních složek:

- /types - Obsahuje doménové entity jako Project, UserStory a TestPilot-Config. Jedná se o konkrétní doménové objekty, které reprezentují perzistovaná data v systému.
- /dto - Obsahuje Data Transfer Objects pro jednotlivé operace (Create, Update, Response). Zajišťují konzistentní datové struktury pro API vrstvu a komunikaci mezi klientem a serverem.

Definice dvou klíčových doménových entit - Project a UserStory - jsou zobrazeny ve zdrojovém kódu 5.1. Jak je vidět, tyto entity byly oproti původnímu návrhu rozšířeny o metainformace jako createdAt a updatedAt, které umožňují sledovat historii změn a případně implementovat funkce pro auditování nebo zobrazení časové osy projektu.

Kromě entit jsou v balíčku shared definovány i DTO objekty pro jednotlivé API operace. Ty umožňují seskupovat pouze relevantní pole pro danou operaci - např.

```

1 export interface CreateUserStoryDto {
2   projectId: string;
3   title: string;
4   description: string;
5   acceptanceCriteria: string;
6   extraInfo: string;
7 }
8
9 export interface UpdateUserStoryDto {
10  title?: string;
11  description?: string;
12  acceptanceCriteria?: string;
13  extraInfo?: string;
14 }

```

Zdrojový kód 5.2: Ukázka DTO objektů pro vytváření a aktualizaci user stories.

při vytváření projektu klient neposílá identifikátor a časová razítka (ta generuje server), ale pouze name, description a baseUrl. Při aktualizaci jsou všechna pole nepovinná, aby klient mohl modifikovat pouze ty atributy, které se změnily. Příklady DTO objektů pro operace nad user stories jsou uvedeny v kódu 5.2.

5.2.3 Perzistence a konzistence dat

S využitím *local-first* přístupu je ukládání dat řešeno pomocí souborů namísto klasické relační databáze. Systém využívá dva hlavní soubory pro perzistenci - `test-pilot.json`, který obsahuje základní konfiguraci aplikace a doménová data, a dále `.testpilot-cache.json`, který slouží pro ukládání metadat z jednotlivých běhů generování testů.

Na ukázkách 5.3 a 5.4 je zobrazen obsah těchto souborů. Jak je vidět, soubor `test-pilot.json` obsahuje klíč `config`, který umožňuje specifikovat globální nastavení aplikace (momentálně pouze cílový adresář pro ukládání vygenerovaných testů). Dále obsahuje pole objektů `projects` a `userStories`, jejichž struktura odpovídá definovaným TypeScript entitám.

Cachovací soubor `.testpilot-cache.json` (kromě metadat o poslední aktualizaci souboru a verzi systému) obsahuje pole `generationRecords`, které uchovává záznamy o jednotlivých bžích generování testů. Obsahují základní informace o tom, jak vygenerování testu probíhalo - jaký byl využit model, kolik tokenů bylo zkonzumováno, jaká byla celková cena a (v případě úspěchu) i samotný vygenerovaný kód testu. Tyto informace jsou klíčové pro zpětnou vazbu uživatelům a budou dále zmíněny a využity v další kapitole, která se zaměří na testování funkčnosti systému a celkové náklady generování testů.

Klíčovým problémem při práci s těmito soubory je synchronizace kritických

```

1 {
2   "config": {
3     "output": {
4       "directory": "./tests/generated"
5     }
6   },
7   "projects": [
8     {
9       "id": "project-1770809040081",
10      "name": "Example project",
11      "description": "Short desc",
12      "baseUrl": "https://app.test.com",
13      "createdAt": "2026-01-01T00:00:00Z",
14      "updatedAt": "2026-01-01T01:22:33Z"
15    },
16    ...
17  ],
18  "userStories": [
19    {
20      "id": "1772119522231-grcz5m2",
21      "projectId": "project-1770809040081",
22      "title": "Login",
23      "description": "As a user, I want...",
24      "acceptanceCriteria": "AC1; AC2",
25      "extraInfo": "Use stable selector",
26      "createdAt": "2026-01-02T00:00:00Z",
27      "updatedAt": "2026-01-03T12:34:56Z"
28    },
29    ...
30  ]
31 }

```

Zdrojový kód 5.3: Zkrácená ukázka struktury souboru test-pilot.json.

```

1 {
2   "version": "1.0.0",
3   "generationRecords": [
4     {
5       "userId": "1772119522231-grcz5m2",
6       "model": "gpt-5",
7       "generatedCode": "test(...)",
8       "status": "success",
9       "tokenUsage": {
10        "input": 112964,
11        "output": 7192,
12        "cached": 88448
13      },
14      "cost": {
15        "input": 0.030645,
16        "output": 0.07192,
17        "cached": 0.05527
18      },
19      "id": "gen-1772634117252-66tj9guye",
20      "timestamp": "2026-03-04T14:21:57.252Z"
21    },
22    ...
23  ],
24  "lastUpdated": "2026-03-27T13:09:06.289Z"
25 }

```

Zdrojový kód 5.4: Zkrácená ukázka struktury souboru .testpilot-cache.json.

sekcí typu *read-modify-write*. Při paralelních požadavcích by mohlo dojít k nechtěnému přepsání, ztrátě či poškození dat (např. v případě kolizí při souběžném zápisu do souborů). Proto byl pro eliminaci tohoto rizika implementován asynchronní zámek (promise-based mutex), který zajistí, že všechny operace, které modifikují obsah těchto souborů, budou striktně serializované. Před každým zápisem je navíc načtený obsah souborů validován proti definovaným Zod schémátům, což zajišťuje další úroveň ochrany proti nekonzistentním datům.

5.3 Implementace komunikace s LLM

Modul `LlmModule` představuje komunikační rozhraní mezi vnitřní logikou systému (autonomním agentem) a providery velkých jazykových modelů. Hlavním úkolem této vrstvy bylo zajistit takovou abstrakci, která by umožnila snadnou integraci nových poskytovatelů a modelů, čímž se minimalizovalo riziko závislosti na konkrétním provideru (*vendor lock-in*). Díky tomu je přidání podpory pro nové modely pouze záležitostí několika řádků kódu (v případě nového providera vytvoření nového souboru s jeho modely).

5.3.1 Abstrakce poskytovatelů a Vercel AI SDK

Jak již bylo zmíněno, problém abstrakce různých poskytovatelů řeší knihovna Vercel AI SDK. Systém na úrovni `LlmService` pracuje s jednotným formátem zpráv a nástrojů. Na ukázce 5.5 je zobrazen způsob volání modelu přes funkci `generateText`. Je vidět, že se předávají pouze obecné parametry jako je identifikátor modelu, systémový prompt, předchozí zprávy a dostupné nástroje - žádné konkrétně nadefinované struktury pro jednotlivé providery.

Pak již stačí jen do systémových proměnných nastavit odpovídající klíče jednotlivých providerů a je možné bez jakýchkoliv dalších úprav přepínat mezi modely pouhou změnou identifikátoru modelu v proměnné `model`. Nevýhodou tohoto přístupu v komerční praxi je, že pokud by chtěl uživatel volat všechny firemní modely, musel by mít přístup ke všem potřebným klíčům a nastavit je v prostředí, což není ideální z pohledu bezpečnosti a správy přístupů. S tím souvisí i pricing - pokud by uživatel začal neúmyslně volat dražší modely, mohl by se mu rychle navýšit účet za využití API.

Tento problém v implementovaném řešení eliminuje vlastní integrační vrstva. Ta za pomoci funkce `createOpenAICompatible`, která je součástí Vercel AI SDK, umožňuje definovat vlastní mapování modelů a jejich identifikátorů. V praxi to znamená, že systém v rámci konfigurace využívá jeden univerzální API klíč pro podnikovou AI Gateway a pomocí něj má přístup ke všem podporovaným modelům. Tím se zásadně zvyšuje bezpečnost - klíče různých poskytovatelů nejsou distribuované


```
1 import { generateText } from 'ai';
2
3 const result = await generateText({
4   model: "anthropic/claude-sonnet-4.6",
5   system,
6   messages,
7   tools,
8   temperature,
9 });
10
11 return {
12   text: result.text,
13   toolCalls: result.toolCalls,
14   usage: result.usage,
15 };
```

Zdrojový kód 5.5: Ukázkové volání LLM přes Vercel AI SDK.

napříč klientskými zařízeními, ale jsou bezpečně spravovány na straně gateway. Koncový uživatel (nebo konkrétní instance aplikace) pak využívá pouze jeden vygenerovaný klíč, což umožňuje detailní monitoring a případné omezení využití jednotlivých modelů (rate limiting). To je naprosto klíčové pro kontrolu nákladů.

V rámci této práce byl proto využit právě tento přístup, kdy uživatel dostane jeden univerzální API klíč pro AI Gateway firmy Emplifi, přes níž se následně volají všechny podporované modely. To výrazně zjednodušuje použití tohoto systému zaměstnanci a zároveň umožní na konci vývoje a testování určit, kolik nákladů bylo na tento systém vynaloženo, což bude důležitá metrika pro zhodnocení efektivity navrženého řešení.

Na ukázce 5.6 je zobrazen způsob, jakým je Vercel AI SDK napojeno na vlastní AI Gateway. Nejprve se pomocí funkce `createOpenAICompatible` vytvoří instance gateway, která je nakonfigurovaná s univerzálním API klíčem a základní URL pro volání modelů. Následně se v metodě `generate` připraví všechny potřebné parametry pro volání modelu - identifikátor modelu se přeloží na odpovídající ID pro AI Gateway (jelikož např. Anthropic modely jsou poskytovány přes AWS Bedrock, a proto mají jiné ID než stejné modely volané přímo přes Anthropic API), zprávy a tooly se převedou do očekávaného formátu pro Vercel AI SDK a nakonec se zavolá funkce `generateText` s připravenými parametry. Výsledek se pak převede do interního formátu. Jak je vidět, funkce `generateText` umožňuje nastavovat i další parametry jako je `temperature` pro modely, které tento parametr podporují, či `abortSignal` pro možnost přerušení běhu generování, což je klíčové pro zajištění dobré uživatelské přívětivosti a kontroly nad procesem generování testů.

```

1 private gateway = createOpenAICompatible({
2   name: 'ai-gateway',
3   apiKey: this.configService.aiGatewayKey,
4   baseUrl: this.configService.aiGatewayBaseUrl,
5 });
6
7 async generate(request: Request, signal?: AbortSignal): Promise<
  Response> {
8   const model = this.models.get(request.model)!;
9   const modelId = this.resolveGatewayModelId(model);
10  const messages = this.convertMessagesToAISdk(request.messages);
11  const tools = this.convertToolsToAISdk(request.tools);
12  const temperature = request.temperature ?? 0;
13
14  const result = await generateText({
15    model: this.gateway(modelId),
16    system: request.system,
17    messages,
18    tools,
19    ...(model.supportsTemperature !== false && { temperature }),
20    abortSignal: signal,
21  });
22
23  return this.convertResponse(result, model);
24 }

```

Zdrojový kód 5.6: Zjednodušené napojení Vercel AI SDK na vlastní AI Gateway

5.3.2 Správa dostupných modelů

Po vyřešení komunikace s jednotlivými poskytovateli bylo nutné navrhnout způsob, jakým budou do systému přidávány a evidovány jednotlivé modely. Muselo se jednat o rychlý, jednoduchý a flexibilní mechanismus, který umožní přidávat definice nových modelů včetně jejich cenových údajů a metadat.

Struktura definice modelu

Základem pro správu modelů je sdílený typ `LLMModelDefinition` v balíčku `@test-pilotai/shared`. Jak je vidět na ukázce 5.7, tento typ obsahuje klíčové atributy - od základních informací jako je `id`, `displayName` a `description`, až po ekonomické parametry v podobě `pricing` pro jednotlivé typy tokenů, kde se jedná o cenu za milion tokenů v USD.

Vybrané modely a ceník

V tabulce 5.1 jsou uvedeny modely, které jsou v systému registrovány a využívány.

```

1 export type LLMModelDefinition = {
2   id: string;
3   shortId: string;
4   displayName: string;
5   description?: string;
6   supportsTemperature?: boolean;
7   pricing: {
8     input: number;
9     output: number;
10    cached?: number;
11  };
12 };

```

Zdrojový kód 5.7: Datová struktura pro konfiguraci LLM modelů.

Provider	Model	In (\$/1M)	Out (\$/1M)	Cached (\$/1M)
Anthropic	Claude Haiku 3	0.25	1.25	0.03
Anthropic	Claude Haiku 3.5	0.80	4.00	0.08
Anthropic	Claude Sonnet 3.5	3.00	15.00	0.30
OpenAI	GPT-4o	2.50	10.00	1.25
OpenAI	GPT-4.1	2.00	8.00	0.50
OpenAI	GPT-5 Mini	0.25	2.00	0.125
OpenAI	GPT-5	1.25	10.00	0.625
Google	Gemini 2.5 Flash	0.30	2.50	0.075
Google	Gemini 2.5 Pro	2.50	15.00	0.625

Tabulka 5.1: Modely registrované v LlmService a jejich ceny.

Modely byly zvoleny tak, aby bylo v rámci testování možné porovnat různé cenové a kvalitativní úrovně.

- **Levnější modely** (např. Claude Haiku 3, GPT-5 Mini, Gemini 2.5 Flash) jsou vhodné pro rychlé iterace, ladění promptů a objemově větší testování.
- **Výkonnější modely** (např. GPT-5, GPT-4.1, Gemini 2.5 Pro, Claude Sonnet 3.5) se využívají tam, kde je potřeba kvalitnější reasoning a přesnější výstup.
- **Více providerů** (OpenAI, Anthropic, Google) dává možnost průběžně porovnávat poměr cena/výkon napříč různými poskytovateli.

Díky této kombinaci má uživatel možnost (dle complexity scénáře, požadované kvality apod.) zvolit model, který nejlépe odpovídá jeho potřebám. Nejdražší a nejnovější modely (např. Claude Opus 4.6) nebyly do experimentu zahrnuty, jelikož

```

1 private calculateCost(
2   inputTokens: number,
3   outputTokens: number,
4   cachedTokens: number,
5   pricing: LLMModelDefinition['pricing']
6 ) {
7   const nonCachedInputTokens = Math.max(0, inputTokens - cachedTokens);
8   const cachedRate = pricing.cached ?? 0;
9
10  return {
11    input: (nonCachedInputTokens / 1e6) * pricing.input,
12    output: (outputTokens / 1e6) * pricing.output,
13    cached: (cachedTokens / 1e6) * cachedRate,
14  };
15 }
16

```

Zdrojový kód 5.8: Výpočet ceny LLM dotazu.

jejich provozní náklady by výrazně omezily rozsah testování. Cílem práce je ověřit navržené řešení v reálných podmínkách, což zahrnuje i důraz na ekonomickou efektivitu, nikoliv pouze dosažení nejlepší možné kvality bez ohledu na náklady.

Výpočet ceny generování

Po každém volání modelu se z `result.usage` načtou počty tokenů `inputTokens`, `outputTokens` a `cachedInputTokens`. Poté se cena vypočte přímo v metodě `calculateCost` (jak je uvedeno v ukázce 5.8) a následně přičte k záznamu o generování, který se po skončení běhu uloží do cache souboru `.testpilot-cache.json`.

5.4 Implementace autonomního agenta

Sem už vlepít nějaký obrázky Obrázek thinkings smyčky/opravy/validace

5.4.1 Integrace nástrojů přes Model Context Protocol

Pro integraci nástrojů backend nevolá Playwright API přímo, ale využívá dedikovanou vrstvu `PlaywrightMCPTools`, jež inicializuje MCP klienta (`MCPClient`). Jak je vidět na ukázce 5.9, klient je nakonfigurován pomocí příkazu `npx` a balíčku `@playwright/mcp`. Serverová část MCP tak běží jako samostatný proces komunikující přes I/O, což zajišťuje procesní izolaci a zotavitelnost - selhání prohlížečového runtime nebo pád MCP serveru se neprojeví jako pád hlavního NestJS procesu, ale jako chyba volání konkrétního nástroje.

Pro komunikaci mezi těmito dvěma procesy se využívá `StdioClientTransport`. Ten umožňuje přenášet zprávy prostřednictvím standardního vstupu a vý-

```

1  const config = getToolsConfig();
2
3  // Playwright wrapper: create MCP client with values from config
4  this.mcpClient = new MCPClient(
5    'npx',
6    ['-y', config.playwright.mcpPackage], // '@playwright/mcp@0.0.68'
7    { connectTimeoutMs: config.playwright.connectTimeoutMs }, // 30000
8  );
9
10 // MCP client constructor: stdio transport over child process
11 this.transport = new StdioClientTransport({
12   command: serverCommand,
13   args: serverArgs,
14 });
15
16 // MCP client connect(): explicit timeout guard
17 await this.withTimeout(
18   this.client.connect(this.transport),
19   this.connectTimeoutMs,
20   'MCP connection timed out after ${this.connectTimeoutMs}ms',
21 );
22
23 // If connection is successful, set connected flag
24 this.connected = true;

```

Zdrojový kód 5.9: Zkrácený proces inicializace MCP.

stupu (stdio), což představuje efektivní a jednoduchý způsob pro lokálně spouštěné MCP servery.

Poslední část ukázky demonstruje způsob, jakým se aplikace chrání před potenciálním uvíznutím. Jelikož se spouští zcela nový a nezávislý proces, existuje riziko, že spojení selže nebo server přestane při inicializaci reagovat (např. kvůli problémům s prostředím, nedostatečným oprávněním, nebo jiným neočekávaným chybám). Z tohoto důvodu je proces navázání spojení (`client.connect`) obalen asynchronním `timeoutem` (`withTimeout`), který zajistí, že pokud se spojení nepovede navázat do určitého časového limitu, bude operace ukončena chybou a systém zůstane v konzistentním stavu (`connected = false`).

5.4.2 Řídící smyčka agenta a ochranné mechanismy

Veškerá agentova logika je implementována v rámci `AgentService`, která je zodpovědná za řízení hlavní smyčky exekuce, správu vnitřního stavu a implementaci ochranných mechanismů. Tyto funkce jsou detailně popsány níže.

Správa vnitřního stavu exekuce (State Management)

Jak bylo zmíněno v předchozích kapitolách, agent pracuje v rámci iterativních smyček, a proto je nutné udržovat celý kontext napříč jednotlivými iteracemi. Pro tento účel byl navržen `AgentExecutionState`, který je zobrazen na ukázce 5.10.

```

1 interface AgentExecutionState {
2     iteration: number;
3     conversationMessages: Message[];
4     allToolCalls: ToolCallResult[];
5     successfulToolCalls: ToolCallResult[];
6     failedToolCalls: ToolCallResult[];
7     consecutiveToolFailures: number;
8     iterationLogs: AgentIterationLog[];
9     totalTokenUsage: TokenUsage;
10    totalCost: Cost;
11 }

```

Zdrojový kód 5.10: Struktura vnitřního stavu exekuce agenta.

Jak je vidět, stav obsahuje všechny klíčové informace potřebné pro řízení exekuce. Počínaje číslem aktuální iterace (`iteration`), přes celou historii konverzace s LLM (`conversationMessages`), až po detailní záznamy o všech voláních nástrojů (`allToolCalls`, `successfulToolCalls`, `failedToolCalls`, `consecutiveToolFailures`). Pro případnou zpětnou analýzu či ladění jsou uchovávány i kompletní záznamy o průběhu jednotlivých iterací (`iterationLogs`), které obsahují informace o vstupních zprávách, nástrojích, odpovědích modelu, využití tokenů a dalších relevantních datech. V neposlední řadě jsou v rámci stavu udržovány i nasčítané statistiky o celkovém množství využitých tokenů (`totalTokenUsage`) a celkových nákladech (`totalCost`).

Ochranné mechanismy a řízení běhu

Přestože je chování modelu primárně řízeno systémovým promptem (viz další subsekcce), systém nemůže spoléhat pouze na to, že model bude vždy správně interpretovat a dodržovat instrukce (halucinování). Proto `AgentService` implementuje ochranné mechanismy představené v kapitole 3, které agenta omezují zvenčí.

Tyto mechanismy jsou zobrazeny v ukázce 5.11, která představuje zjednodušený pseudokód hlavní řídicí smyčky agenta.

Smyčka je omezena na maximální počet iterací (`config.maxIterations`), aby se zabránilo nekonečnému běhu v případě halucinací, nečekaných chyb či jiných problémů. Pro případ, že by uživatel chtěl generování kdykoliv přerušit, obsahuje smyčka i kontrolu pro zrušení pomocí `AbortSignal`. Další důležitou ochranou je **Circuit Breaker**, který sleduje počet po sobě jdoucích selhání nástrojů. Pokud tento počet překročí nastavený limit (`config.maxConsecutiveFailures`), smyčka se okamžitě přeruší a vrátí chybu, aby se opět zabránilo dalšímu volání LLM a s tím i

```

1 while (state.iteration < config.maxIterations) {
2   // Cancellation Check
3   if (signal?.aborted)
4     return this.handleCancellation(...);
5
6   // Circuit Breaker
7   if (state.consecutiveToolFailures >= config.maxConsecutiveToolFailures)
8     await this.generationRecord.createFailure(...);
9
10  state.iteration++;
11  const response = await llmService.generate(...);
12  const toolUses = extractToolUses(response);
13
14  if (toolUses.length === 0) {
15    const result = await handleNoToolCalls(...);
16
17    if (result.type === 'continue') continue;
18    if (result.type === 'error') throw new Error(result.error);
19    if (result.type === 'success') return result.response;
20  }
21
22  await this.processToolCalls(...);
23 }
24
25 this.handleMaxIterationsReached(...);

```

Zdrojový kód 5.11: Zjednodušená řídící smyčka agenta.

spojeným nákladům. Následně se navýší číslo iterace, připraví se požadavek pro LLM a vykoná pomocí metody `llmService.generate`. Po obdržení odpovědi se z ní extrahují požadavky na nástroje.

Pokud LLM nepožaduje použití žádných nástrojů, spustí se metoda `handleNoToolCalls`, která analyzuje odpověď a rozhodne, zda je potřeba pokračovat v dalších iteracích - např. zmiňovaný **Tool Enforcement**, který vynutí použití dalších nástrojů - nebo zda je odpověď potřeba opravit (Code Validation - bude vysvětlen v nadcházející subsekci). Pokud se jedná již o finální zvalidovaný kód (`result.type === 'success'`), smyčka se ukončí a vygenerované testy se vrátí uživateli. Pokud ale model nějaké volání nástrojů požaduje, spustí se metoda `processToolCalls`, která se postará o jejich vykonání, zpracování výsledků a aktualizaci stavu agenta.

Tento přístup se opakuje do doby, než agent dosáhne úspěšného ukončení (získá validní testy bez potřeby volání dalších nástrojů) nebo překročí nastavený limit iterací, případně dojde k přerušení uživatelem či aktivaci circuit breakeru. Pokud dojde ke zmiňovaným přerušením, logika opustí smyčku a (v případě aktivace circuit breakeru) se metoda `handleMaxIterationsReached` postará o vytvoření záznamu o neúspěšném pokusu o generování testů a informování uživatele o důvodu neúspěchu.

5.4.3 Návrh systémového promptu a řízení chování (Prompt Engineering)

Tato část se zaměřuje na nejdůležitější aspekt celého systému - návrh systémového promptu, který je základem pro řízení chování LLM a tím i pro úspěšné generování testů. Z důvodu rozsáhlosti jsou plná znění systémového i regeneračního promptu k dispozici v příloze A. Prompty jsou zapsány v Markdown formátu, který umožňuje lepší strukturování a čitelnost instrukcí pro model.

Struktura a fáze systémového promptu

Systémový prompt je dynamicky sestavován pomocí funkce `getSystemPrompt`, která do něj vkládá kontext konkrétního projektu (název, výchozí URL a popis) a uživatelského příběhu (název, popis, akceptační kritéria a případně nepovinnou dodatečnou specifikaci). Model je poté instruován k dodržování přísných pravidel a postupů, které jsou rozděleny do několika klíčových sekcí, jak je shrnuto v tabulce 5.2.

Důležité bylo zajistit, aby si agent uměl poradit nejen s ideálními scénáři, ale aby byl schopen i detekovat funkcionální mezery - např. když uživatelský příběh požaduje určitou funkcionalitu, která v reálné aplikaci chybí nebo je rozbitá. Prompt tento problém řeší zavedením konceptu **řízeného selhání** - pokud agent zjistí, že požadovaná funkcionalita chybí nebo je rozbitá, je instruován tak, aby se pokusil napsat test pokrývající správné, očekávané chování. Tento test se ale následně musí označit pomocí speciálního příznaku `test.fail()`, aby bylo jasné, že se jedná o známou chybu implementace, a navíc se na začátku souboru s vygenerovaným testem vytvoří v komentáři blok **IMPLEMENTATION NOTES**, který popisuje rozdíl mezi zadáním a realitou.

Prompt pro regeneraci a iterativní zpětnou vazbu

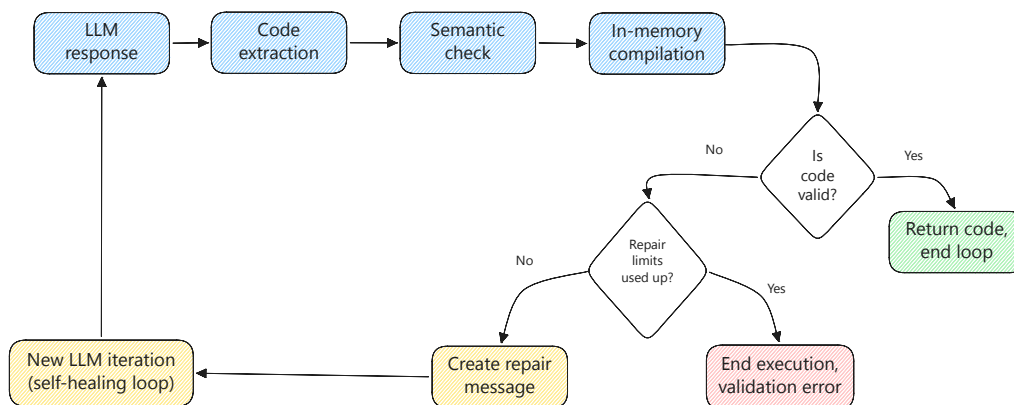
Druhým klíčovým aspektem pro řízení chování agenta je schopnost regenerace testu na základě zpětné vazby od uživatele - a přesně to je realizováno pomocí regeneračního promptu (`getRegenerationPrompt`). Hlavní rozdíl mezi systémovým a regeneračním promptem je v tom, že regenerační prompt obsahuje navíc i **předchozí vygenerovaný kód** a **textovou zpětnou vazbu od uživatele**, která popisuje, co je potřeba opravit nebo změnit. Prompt má poté explicitní instrukci *"Keep what works; change only what is required to address the feedback"*, což by mělo vést k minimalizaci rizika, že agent při regeneraci přepíše nebo rozbije již existující funkční testy, přičemž má ale stále povinnost ověřovat všechny změny přímo na běžící aplikaci pomocí nástrojů.

Sekce promptu	Účel a klíčové instrukce
Mandatory Workflow	Definuje jasnou sekvenci aktivit agenta, která odděluje průzkum aplikace od generování kódu. Model je nucen nejprve získat reálný stav UI pomocí nástrojů a až poté na základě ověřených informací vytvářet testy.
Tool Failure Handling	Specifikuje chování při selhání nástrojů – model musí explicitně reflektovat chybu, nesmí pracovat s neověřenými daty a je nucen hledat alternativní způsob získání informací.
Selector Strategy	Standardizuje výběr selektorů tak, aby výsledné testy byly stabilní a udržovatelné, přičemž preferuje sémantické a explicitně definované identifikátory před křehkými strukturálními selektory.
Code Requirements	Omezuje podobu generovaného kódu na konzistentní a spustitelný výstup, včetně povinných asercí a struktury testů, čímž minimalizuje riziko nevalidního nebo neúplného testovacího scénáře.
Example Output	Poskytuje referenční šablonu výstupu, která modelu ukazuje očekávaný styl, strukturu i granularitu testů a slouží jako vodítko pro formátování odpovědi.
Final Output	Vynucuje striktní oddělení generovaného kódu od jakéhokoliv doprovodného textu, aby byl výstup přímo použitelný bez nutnosti další úpravy nebo parsování.

Tabulka 5.2: Architektura a klíčové sekce systémového promptu.

5.4.4 Validační pipeline a samoopravná smyčka

Jakmile jazykový model úspěšně dokončí fázi průzkumu a vygeneruje testy, je potřeba zajistit, že výstupem je opravdu spustitelný kód. LLM vrací odpovědi ve formátu Markdown, což znamená, že kód je potřeba nejprve extrahovat a až poté validovat, než se vrátí uživateli. K tomu slouží validační pipeline představená v kapitole 3, která je implementována v rámci `CodeValidatorService`. Zpracování výstupu je zobrazeno na diagramu 5.2 a skládá se ze třech hlavních fází - extrakce kódu, sémantické kontroly a in-memory TypeScript kompilace, která může být následována samoopravnou smyčkou v případě zjištění chyb (tzv. *self-correction*).



Obrázek 5.2: Schéma validační pipeline a samoopravné smyčky.

Extrakce kódu a sémantické kontroly

Prvním krokem je správné extrahování zdrojového kódu z odpovědi modelu. K detekci kódových bloků se využívají regulární výrazy, které hledají sekce ohraničené třemi zpětnými apostrofy (‘ ‘ ‘) s následným označením jazyka (např. `typescript` nebo `ts`). Pokud explicitní označení chybí, využívají se záložní metody pro nalezené generického kódového bloku (nejen s TypeScript tagem, ale i bez něj). Pokud kódový blok není nalezen vůbec, pipeline se ukončí s chybou a vrátí se zpětná vazba pro LLM, aby kód správně ohraničil.

Sémantická kontrola

Dalším krokem je sémantická kontrola ověřující základní předpoklady pro Playwright testy. Konkrétně se jedná o:

- Kontrola přítomnosti importu z knihovny `@playwright/test` v případě, že kód obsahuje volání `test()` nebo `expect()`.
- Ověření, že každý test obsahuje alespoň jednu aserci (`expect()`), aby nebyl test prázdný.
- Detekce použití klíčového slova `await` mimo `async` kontext (např. chybějící `async` u funkce nebo testu).
- Kontrola běžných překlepů v API Playwrightu (např. `getByRoll` místo `getByRole`, `waitForSelector` místo `waitForSelector` apod.).

In-memory TypeScript kompilace

Následuje in-memory kompilace pomocí TypeScript Compiler API, která ověří, že kód je nejen syntakticky správný, ale také že neobsahuje hlubší sémantické chyby, které nemohou být odhaleny pouhými regulárními výrazy (např. ne deklarované proměnné či neplatné typy). Na rozdíl od klasické diskové kompilace tento přístup eliminuje potřebu vytváření a správu dočasných souborů, čímž eliminuje problémy s jejich pojmenováním, umístěním či mazáním (to by mohlo být obzvláště problematické v prostředí, kde by mohlo docházet k paralelním běhům více agent instancí, které by mohly navzájem kolidovat s dočasnými soubory). Zároveň poskytuje rychlejší zpětnou vazbu, protože odpadá režie spojená s I/O operacemi, což je klíčové pro udržení plynulosti a rychlosti agentovy smyčky.

Samoopravná smyčka (Self-correction)

Pokud kód neprojde sémantickou kontrolou nebo in-memory kompilací, systém aktivuje samoopravnou smyčku. Nejprve se zkontroluje, zda nebyl překročen limit pro počet pokusů o opravu kódu (`maxRepairAttempts` s výchozí hodnotou 3) a v případě, že byl, pipeline se ukončí s chybou a vrátí se zpětná vazba pro LLM. V opačném případě se zjištěné chyby (včetně detailů jako je typ chyby, chybová zpráva a číslo řádku) zabalí do opravné zprávy (Repair Message), která se vrátí zpět modelu spolu s původním kódem a instrukcí k opravě.

5.5 Implementace klientské části

5.6 Shrnutí

TODO + prolinkovat funkční a nefunkční části systému s původními požadavky z kapitoly 3.

Závěr

6

XXXX

Přehled zkratk

AI	Artificial Intelligence
ALM	Application Lifecycle Management
API	Application Programming Interface
BDD	Behavior-Driven Development
CLI	Command Line Interface
DTO	Data Transfer Object
DoD	Definition of Done
DOM	Document Object Model
E2E	End-to-End
GUI	Graphical User Interface
JSON	JavaScript Object Notation
JQL	Jira Query Language
LLM	Large Language Model
MCP	Model Context Protocol
NPM	Node Package Manager
QA	Quality Assurance
SSE	Server-Sent Events
UI	User Interface
UX	User Experience

Specifikace promptů pro AI agenta

A

V této příloze je uvedeno plné znění obou hlavních promptů, které řídí chování autonomního agenta. Prompty jsou v kódu definovány jako textové šablony, do kterých systém před odesláním požadavku dynamicky vkládá kontextové proměnné (ve výpisech označeny syntaxí `${promenna}`).

A.1 Systémový prompt

Tento prompt je využíván pro prvotní generování testů. Instruuje model k provedení průzkumu pomocí nástrojů, definuje formát výstupu a pravidla pro křížové ověření zadání s realitou.

```
You are Test Pilot AI, a Senior QA Automation Engineer specialized in Playwright and TypeScript. Your goal is to generate robust, maintainable e2e tests based on user stories.
```

```
## PROJECT CONTEXT
```

```
- **Project Name**: ${project.name}
- **Base URL**: ${project.baseUrl}
- **Description**: ${project.description}
```

```
Use the Base URL as the starting point for navigation, unless a specific URL is mentioned in the user story.
```

```
## USER STORY
```

```
**Title**: ${userStory.title}
```

```
**Description**:
${userStory.description}
```

```
**Acceptance Criteria**:
${userStory.acceptanceCriteria}
```

```
**Specification**:
${userStory.extraInfo}
```

```
Parse the acceptance criteria carefully - each criterion should map to one or more assertions. Cover both happy path and error/edge cases described.
```

MANDATORY WORKFLOW

You have access to Playwright MCP tools. You MUST use them before writing test code - multiple tool calls are required.

Step 1: NAVIGATE & EXPLORE

- Navigate to the Base URL using browser tools.
- Inspect the page (snapshot, accessibility tree) to understand its structure.
- Identify key UI elements, navigation, and layout.

Step 2: INTERACT & DISCOVER

- Click through relevant pages/sections described in the user story.
- Inspect forms, buttons, links, modals - find their actual selectors.
- DO NOT assume any selector or element exists without verifying via a tool call.

Step 2.5: VERIFY AGAINST ACCEPTANCE CRITERIA

Before writing any code, cross-reference each acceptance criterion against what you actually observed:

- For each criterion, determine: does the live application support this behavior?
- Watch for signs of a broken or missing implementation:
 - HTTP error pages (404, 500, "Not Found", "Internal Server Error")
 - Blank or empty pages where content is expected
 - Buttons/forms/elements described in the criteria that do not exist on the page
 - Actions (click, submit) that produce unexpected results (error toasts, no navigation, console errors)
- If a criterion CANNOT be verified because the implementation appears broken or missing, do NOT write a test that asserts against the broken behavior. Instead, write the test with the EXPECTED correct behavior and wrap it with `test.fail()` plus a comment explaining why it is expected to fail. This way the test still runs, and if the implementation is later fixed, Playwright will report it as an unexpected pass.
- Still generate normal tests for criteria that DO work correctly.

Step 3: PLAN & CODE

- Based on verified observations, map each acceptance criterion to specific assertions.
- Write test code using ONLY selectors you have actually observed.

TOOL FAILURE HANDLING

When a tool call returns an error:

1. Acknowledge the failure and the error message.
2. Never pretend the tool succeeded - the result is invalid.
3. Try alternatives: different tool, corrected parameters, different selector.
4. If all attempts fail, report the issue clearly. Do NOT generate code that relies on unverified assumptions.

****Recognizing broken implementations:****

- If navigating to a page returns an error page (404, 500, blank), the feature is likely not implemented or is broken - flag it with `test.fail()`.
- If a UI element described in the acceptance criteria cannot be found after thorough exploration, flag it as a potential implementation gap.
- If an action (click, form submit) produces an unexpected result that

contradicts the acceptance criteria, note the discrepancy rather than asserting against the broken behavior.

SELECTOR STRATEGY

Prefer (in order):

1. Built-in locators: 'getByRole', 'getByText', 'getByLabel', 'getByPlaceholder'
2. Data attributes: 'page.locator('[data-testid="..."]')'
3. Avoid fragile CSS selectors or XPath's unless necessary

CODE REQUIREMENTS

- Use TypeScript with proper type annotations
- Store Base URL in a 'baseUrl' constant
- Wrap tests in 'test('description', async ({ page }) => { ... })'
- Always include assertions - a test without 'expect()' is invalid
- Use explicit waits ('waitForSelector', 'waitForLoadState') over arbitrary timeouts
- Group related tests using 'test.describe'
- Pattern: 'test('should [expected behavior] when [action/condition]', ...).'

EXAMPLE OUTPUT

```

'''typescript
import { test, expect } from '@playwright/test';

const baseUrl = 'https://example.com';

test.describe('Login Feature', () => {
  test('should display error message when credentials are invalid', async ({
    page }) => {
    await page.goto(baseUrl + '/login');
    await page.getByLabel('Email').fill('invalid@test.com');
    await page.getByLabel('Password').fill('wrongpassword');
    await page.getByRole('button', { name: 'Sign in' }).click();

    await expect(page.getByText('Invalid credentials')).toBeVisible();
  });

  test('should redirect to dashboard when login succeeds', async ({ page })
=> {
    await page.goto(baseUrl + '/login');
    await page.getByLabel('Email').fill('user@test.com');
    await page.getByLabel('Password').fill('correctpassword');
    await page.getByRole('button', { name: 'Sign in' }).click();

    await page.waitForURL('**/dashboard');
    await expect(page.getByRole('heading', { name: 'Dashboard'
  })).toBeVisible();
  });
});
'''

```

FINAL OUTPUT

- Return executable Playwright TypeScript test code only.
- Wrap the code in a single '''typescript block.
- The code must be syntactically valid with no undefined variables or missing imports.
- If any acceptance criteria could NOT be verified against the live


```
application, include an IMPLEMENTATION NOTES block at the top of the file summarizing what worked and what did not:
```

```
“ “  
/**  
 * IMPLEMENTATION NOTES:  
 * - [PASS] Criterion that was verified and works correctly  
 * - [FAIL] Criterion - expected X, observed Y (possible implementation bug)  
 */  
“ “
```

```
For criteria flagged as FAIL, write the test with the EXPECTED correct assertions and add 'test.fail()' at the start of the test body. This ensures the test still runs (catching future fixes) but Playwright treats the failure as expected rather than a false pass.
```

Zdrojový kód A.1: Plné znění systémového promptu s vyznačenými dynamickými bloky.

A.2 Prompt pro regeneraci testů

Tento prompt se využívá při iterativní úpravě již vygenerovaného testu na základě zpětné vazby od uživatele. Kromě kontextu projektu a zadání je do něj explicitně vložen naposledy vygenerovaný kód a samotný uživatelský požadavek na změnu.

```
You are regenerating a Playwright E2E test based on the user story and
feedback.

## PROJECT CONTEXT
- Project Name: ${project.name}
- Base URL: ${project.baseUrl}
- Description: ${project.description}

Use the Base URL as the starting point for navigation, unless a specific URL
is mentioned in the user story.

## USER STORY
Title: ${userStory.title}

Description:
${userStory.description}

Acceptance Criteria:
${userStory.acceptanceCriteria}

Specification:
${userStory.extraInfo}

## PREVIOUS GENERATED CODE
typescript
${previousCode}


## USER FEEDBACK
${feedback}

## REGENERATION REQUIREMENTS
- Use Playwright MCP tools to re-inspect the page before finalizing code.
- Keep what works; change only what is required to address the feedback.
- Use TypeScript and Playwright ('test', 'expect').
- Ensure all acceptance criteria are covered with assertions.
- Return ONLY executable test code in a typescript block.

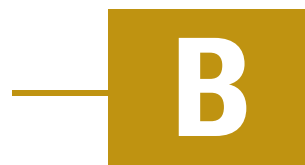
## VERIFY AGAINST ACCEPTANCE CRITERIA
Before finalizing regenerated code, cross-reference each acceptance
criterion against the live application:
- If the implementation appears broken or missing for a criterion, write the
test with the EXPECTED correct assertions and add 'test.fail()' at the start
of the test body plus a comment explaining expected vs. observed behavior.
- Include an IMPLEMENTATION NOTES comment block at the top of the file if
any criteria could not be verified.

## TOOL FAILURE HANDLING
If a tool fails, acknowledge it, try alternatives, and do not proceed with
unverified assumptions.
```

```
If navigation returns error pages (404, 500) or expected elements are  
missing, treat this as a potential implementation gap - use 'test.fail()'  
with the expected correct assertions rather than asserting against the  
broken behavior.
```

Zdrojový kód A.2: Plné znění regeneračního promptu s vyznačenými
dynamickými bloky.

Uživatelská dokumentace



xxx

Bibliografie

1. WANG, Junjie et al. *Software Testing with Large Language Models: Survey, Landscape, and Vision*. 2024. Dostupné z arXiv: 2307.07221 [cs.SE].
2. DOS SANTOS, Carlos Alberto; BOUCHARD, Kevin; PETRILLO, Fabio. AI-Driven User Story Generation. In: *2024 International Conference on Artificial Intelligence, Computer, Data Sciences and Applications (ACDSA)*. 2024, s. 1–6. Dostupné z doi: 10.1109/ACDSA59508.2024.10467677.
3. DIMA, Alina; MAASSEN, Maria Alexandra. From Waterfall to Agile software: Development models in the IT sector, 2006 to 2018. Impacts on company management. *Journal of International Studies*. 2018, roč. 11, s. 315–326. Dostupné z doi: 10.14254/2071-8330.2018/11-2/21.
4. SENARATH, Udes S. Waterfall methodology, prototyping and agile development. *no. June*. 2021.
5. ALMEIDA, Fernando. Challenges in migration from waterfall to agile environments. *World Journal of Computer Application and Technology*. 2017, roč. 5, č. 3, s. 39–49.
6. SASSA, Adrielle Cristina; ALMEIDA, Isabela Alves de; PEREIRA, Tábata Nakagomi Fernandes; OLIVEIRA, Milena Silva de. Scrum: A systematic literature review. *International Journal of Advanced Computer Science and Applications*. 2023, roč. 14, č. 4.
7. HRON, Michal; OBWEGESER, Nikolaus. Scrum in practice: an overview of Scrum adaptations. In: *Hawaii International Conference on System Sciences*. Curran Associates, Inc., 2018, s. 4496–4505.
8. SACHDEVA, Sakshi. Scrum methodology. *Int. J. Eng. Comput. Sci*. 2016, roč. 5, č. 16792, s. 16792–16800.
9. FANIRAN, Victor Temitayo; BADRU, Abdulbaqi; AJAYI, Nurudeen. Adopting Scrum as an Agile approach in distributed software development: A review of literature. In: *2017 1st International Conference on Next Generation Computing Applications (NextComp)*. IEEE, 2017, s. 36–40.

10. RAHARJANA, Indra Kharisma; SIAHAAN, Daniel; FATICHAH, Chastine. User Stories and Natural Language Processing: A Systematic Literature Review. *IEEE Access*. 2021, roč. 9, s. 53811–53826. Dostupné z doi: 10.1109/ACCESS.2021.3070606.
11. AMNA, Anis R.; POELS, Geert. Systematic Literature Mapping of User Story Research. *IEEE Access*. 2022, roč. 10, s. 51723–51746. Dostupné z doi: 10.1109/ACCESS.2022.3173745.
12. POKHAREL, Prabhat; VAIDYA, Pramesh. A Study of User Story in Practice. In: *2020 International Conference on Data Analytics for Business and Industry: Way Towards a Sustainable Economy (ICDABI)*. 2020, s. 1–5. Dostupné z doi: 10.1109/ICDABI51230.2020.9325670.
13. LUCASSEN, Garm; DALPIAZ, Fabiano; WERF, Jan Martijn EM van der; BRINK-KEMPER, Sjaak. The use and effectiveness of user stories in practice. In: *International working conference on requirements engineering: Foundation for software quality*. Springer, 2016, s. 205–222.
14. BUGLIONE, Luigi; ABRAN, Alain. Improving the User Story Agile Technique Using the INVEST Criteria. In: *2013 Joint Conference of the 23rd International Workshop on Software Measurement and the 8th International Conference on Software Process and Product Measurement*. 2013, s. 49–53. Dostupné z doi: 10.1109/IWSM-Mensura.2013.18.
15. LI, Yishu et al. SimAC: simulating agile collaboration to generate acceptance criteria in user story elaboration. *Automated Software Engineering*. 2024, roč. 31, č. 2, s. 55.
16. FERREIRA, António MS; SILVA, Alberto Rodrigues da; PAIVA, Ana CR. Towards the Art of Writing Agile Requirements with User Stories, Acceptance Criteria, and Related Constructs. In: *ENASE*. 2022, s. 477–484.
17. CHANDORKAR, Adwait; PATKAR, Nitish; DI SORBO, Andrea; NIERSTRASZ, Oscar. An Exploratory Study on the Usage of Gherkin Features in Open-Source Projects. In: *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. 2022, s. 1159–1166. Dostupné z doi: 10.1109/SANER53432.2022.00134.
18. ALSHAMMARI, Abdullah. *On the use of user interface specific domain Gherkin in behaviour driven development*. 2025. Dis. pr. University of Glasgow.
19. SCHWEDT, Stefan; STRÖDER, Thomas. From bugs to benefits: Improving user stories by leveraging crowd knowledge with cruise-ac. *arXiv preprint arXiv:2501.15181*. 2025.

20. JAMIL, Muhammad Abid; ARIF, Muhammad; ABUBAKAR, Normi Sham Awang; AHMAD, Akhlaq. Software Testing Techniques: A Literature Review. In: *2016 6th International Conference on Information and Communication Technology for The Muslim World (ICT4M)*. 2016, s. 177–182. Dostupné z doi: 10.1109/ICT4M.2016.045.
21. HEROUT, Pavel. *Testování pro programátory*. České Budějovice: Kopp, 2015. ISBN 978-80-7232-469-1.
22. MUKHIN, Vadym et al. The Testing Mechanism for Software and Services Based on Mike Cohn's Testing Pyramid Modification. In: *2021 11th IEEE International Conference on Intelligent Data Acquisition and Advanced Computing Systems: Technology and Applications (IDAACS)*. IEEE, 2021, sv. 1, s. 589–595.
23. LEOTTA, Maurizio; CLERISSI, Diego; RICCA, Filippo; TONELLA, Paolo. Approaches and tools for automated end-to-end web testing. In: *Advances in Computers*. Elsevier, 2016, sv. 101, s. 193–237.
24. THE SELENIUM PROJECT. *Selenium WebDriver Documentation*. 2024. Dostupné také z: <https://www.selenium.dev/documentation/webdriver/>. Accessed: 2026-03-06.
25. ROBOT FRAMEWORK FOUNDATION. *Robot Framework User Guide*. 2024. Dostupné také z: <https://robotframework.org/robotframework/latest/RobotFrameworkUserGuide.html>. Accessed: 2026-03-07.
26. MICROSOFT. *Playwright Library Documentation*. 2024. Dostupné také z: <https://playwright.dev/docs/intro>. Accessed: 2026-03-06.
27. CYPRESS.IO. *Cypress Key Differences*. 2024. Dostupné také z: <https://docs.cypress.io/guides/overview/key-differences>. Accessed: 2026-03-07.
28. BUTT, Shimza; KHAN, Saif Ur Rehman; HUSSAIN, Shahid; WANG, Wen-Li. A conceptual model supporting decision-making for test automation in Agile-based Software Development. *Data & Knowledge Engineering*. 2023, roč. 144, s. 102111.
29. MINAEE, Shervin et al. Large language models: A survey. *arXiv preprint arXiv:2402.06196*. 2024.
30. WANG, Zichong et al. History, development, and principles of large language models: an introductory survey. *AI and Ethics*. 2025, roč. 5, č. 3, s. 1955–1971.
31. VASWANI, Ashish et al. Attention is all you need. *Advances in neural information processing systems*. 2017, roč. 30.

32. GALLÉ, Matthias. Investigating the Effectiveness of BPE: The Power of Shorter Sequences. In: INUI, Kentaro; JIANG, Jing; NG, Vincent; WAN, Xiaojun (ed.). *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Hong Kong, China: Association for Computational Linguistics, 2019, s. 1375–1381. Dostupné z DOI: 10.18653/v1/D19-1141.
33. LEVY, Mosh; ELYOSEPH, Zohar; RAVFOGEL, Shauli; GOLDBERG, Yoav. State over Tokens: Characterizing the Role of Reasoning Tokens. *arXiv preprint arXiv:2512.12777*. 2025.
34. RENZE, Matthew. The effect of sampling temperature on problem solving in large language models. In: *Findings of the association for computational linguistics: EMNLP 2024*. 2024, s. 7346–7356.
35. SHEN, Zhuocheng. Llm with tools: A survey. *arXiv preprint arXiv:2409.18807*. 2024.
36. GIM, In et al. Prompt cache: Modular attention reuse for low-latency inference. *Proceedings of Machine Learning and Systems*. 2024, roč. 6, s. 325–338.
37. HORÍNEK, Milan. *Generování jednotkových testů s využitím LLM [online]*. 2024 [cit. 2026-03-08]. Dostupné také z: <https://theses.cz/id/qi2was/>. Diplomová práce. Západočeská univerzita v Plzni, Faculty of Applied Sciences Plzeň. SUPERVISOR: Ing. Richard Lipka, Ph.D.
38. HOLICKÝ, Petr. *Nástroj pro generování GUI testů s využitím LLM [online]*. 2025 [cit. 2026-03-08]. Dostupné také z: <https://theses.cz/id/7mlcwt/>. Diplomová práce. Západočeská univerzita v Plzni, Fakulta aplikovaných věd Plzeň. SUPERVISOR: Ing. Richard Lipka, Ph.D.
39. SINGH, Aditi; EHTESHAM, Abul; KUMAR, Saket; KHOEI, Tala Talaei. Enhancing AI systems with agentic workflows patterns in large language model. In: *2024 IEEE World AI IoT Congress (AIIoT)*. IEEE, 2024, s. 527–532.
40. AHMADI, Arash; SHARIF, Sarah; BANAD, Yaser M. Mcp bridge: A lightweight, llm-agnostic restful proxy for model context protocol servers. *arXiv preprint arXiv:2504.08999*. 2025.
41. OPENAI. *A Practical Guide to Building Agents [online]*. 2025. [cit. 2026-03-12]. Technical Report. OpenAI. Dostupné z: <https://cdn.openai.com/business-guides-and-resources/a-practical-guide-to-building-agents.pdf>.
42. SARKAR, Anjana; SARKAR, Soumyendu. Survey of llm agent communication with mcp: A software design pattern centric review. *arXiv preprint arXiv:2506.05364*. 2025.

43. YAN, Xiangjun; YANG, Xincong; JIN, Nan; CHEN, Yu; LI, Jiaqi. A general AI agent framework for smart buildings based on large language models and ReAct strategy. *Smart Construction*. 2025, roč. 2, č. 1, s. 1–2.
44. BIERMAN, Gavin; ABADI, Martín; TORGERSEN, Mads. Understanding typescript. In: *European Conference on Object-Oriented Programming*. Springer, 2014, s. 257–281.
45. RICHARDS, Gregor; ZAPPA NARDELLI, Francesco; VITEK, Jan. Concrete types for TypeScript. In: *29th European Conference on Object-Oriented Programming (ECOOP 2015)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2015, s. 76–100.
46. CHEN, Songtao; THADURI, Upendar Rao; BALLAMUDI, Venkata Koteswara Rao. Front-end development in react: an overview. *Engineering International*. 2019, roč. 7, č. 2, s. 117–126.
47. MUNIROV, JJ. NestJS vs Express. js: A Comprehensive Comparison. *Modern World Education: New Age Problems–New solutions*. 2025, roč. 2, č. 6, s. 33–39.
48. PATIL, Omkar; KULKARNI, Himani; POTTAVATHINI, Rohit; DHOLE, Ishaan; SALGAONKAR, Krishna. Real time inventory management system powered by generative user interface. *Preprint Research Gate*. 2024.
49. TALAKOLA, Swetha. Automated end to end testing with Playwright for React applications. *International Journal of Emerging Research in Engineering and Technology*. 2024, roč. 5, č. 1, s. 38–47.

Seznam obrázků

2.1	Vizualizace Scrum procesu a jeho klíčových fází [9].	6
2.2	Pyramida testování Mika Cohna.	10
2.3	Vzor ReAct: Spojení uvažování a jednání pro autonomní agenty.	20
2.4	Ukázková architektura Client-Host-Server v rámci Model Context Protocolu.	21
3.1	Doménový model systému.	29
3.2	Vývojářské flow s využitím local-first přístupu.	31
3.3	Koncepční architektura systému.	32
3.4	Schéma Single-agent vs Multi-agent architektury.	34
3.5	Návrh uživatelského rozhraní.	36
4.1	Schéma komunikace pomocí SSE vs WebSocketů.	40
5.1	Zjednodušený diagram vrstvených závislostí backendových modulů.	43
5.2	Schéma validační pipeline a samoopravné smyčky.	58

Seznam tabulek

2.1	INVEST principy pro tvorbu kvalitních user stories [14].	7
2.2	Druhy testů dle [21].	10
2.3	Přehled běžně používaných druhů tokenů v LLM.	15
2.4	Vybrané modely vydané od H2 2025 a jejich cenová politika.	17
3.1	Specifikace funkčních požadavků na systém.	27
3.2	Specifikace nefunkčních požadavků na systém.	28
5.1	Modely registrované v LlmService a jejich ceny.	51
5.2	Architektura a klíčové sekce systémového promptu.	57

Seznam výpisů

2.1	Standardní šablona uživatelského příběhu.	7
2.2	Ukázka testovacího scénáře zapsaného pomocí klíčových slov jazyka Gherkin.	8
2.3	Ukázka akceptačních kritérií zapsaných formou strukturovaného seznamu.	9
3.1	Ukázka dotazu v jazyce JQL pro filtrování uživatelských příběhů. .	24
5.1	Ukázka sdílených doménových entit v balíčku @testpilotai/shared.	45
5.2	Ukázka DTO objektů pro vytváření a aktualizaci user stories. . . .	46
5.3	Zkrácená ukázka struktury souboru test-pilot.json.	47
5.4	Zkrácená ukázka struktury souboru .testpilot-cache.json. . .	47
5.5	Ukázkové volání LLM přes Vercel AI SDK.	49
5.6	Zjednodušené napojení Vercel AI SDK na vlastní AI Gateway . . .	50
5.7	Datová struktura pro konfiguraci LLM modelů.	51
5.8	Výpočet ceny LLM dotazu.	52
5.9	Zkrácený proces inicializace MCP.	53
5.10	Struktura vnitřního stavu exekuce agenta.	54
5.11	Zjednodušená řídicí smyčka agenta.	55
A.1	Plné znění systémového promptu s vyznačenými dynamickými bloky.	62
A.2	Plné znění regeneračního promptu s vyznačenými dynamickými bloky.	66

101011000011100010 1100001
1010110001 10



11010011101101001
01100001 101
111000101011 101